

CEN429 Term Project — Project Guide

CEN429 Secure Programming (formerly CE407) · Fall 2026–2027

In this project you design and build a chosen application **as if it were going through a certification process**: a **C++ console application** plus a **dynamic library (DLL/.so)** that provides its security. The goal is to combine every security technique you learn during the term into **one product** and document all of it in a **security guide** (S0–S17).

Language and tools

C++ (DLL/.so) · CMake/CTest · SQLite ·
[SoftHSM/PKCS#11](#) · OpenSSL

Team

At most 5 students (working alone is fine) · each topic goes to one team only · no team changes after week 3

Midterm checkpoint

Week 7 (30.10.2026) · RAP1 — first half of the product

Final checkpoint

Week 15 (25.12.2026) · RAP2 — full product + test results

At a glance

- **Team:** at most 5 students (you may also work alone); each topic can be taken by only one team. Teams cannot change after week 3 (04.10.2026).
- **Language/tools:** C++ (DLL/.so), CMake/CTest, SQLite, [SoftHSM/PKCS#11](#), OpenSSL.
- **Two deliveries:** **Midterm (RAP1)** — first half of the product; **Final (RAP2)** — full product + test **results**.
- **Heart of the delivery:** alongside the code, a **security guide** written like a certification document (S0–S17).
- All data is **synthetic**; no real secret, key, or personal data may enter the repository.
- A detailed scoring key and evidence templates may be shared on Microsoft Teams; the criteria and points on this page are binding.

1. Calendar

Week	Date	Event
End of week 3	04.10.2026	Last day to choose your topic and team
Week 4	09.10.2026	Project plan approved by the instructor
Week 7	30.10.2026	Midterm demo + interim report (RAP1)
Week 8	31.10–08.11.2026	Quiz-1 (weeks 1–6)
Week 15	25.12.2026	Final demo + final report (RAP2)
Week 16	04–17.01.2027	Quiz-2 (weeks 9–14)

The full weekly content plan is in the [syllabus](#). Any date change is announced in the course classroom.

2. Assessment structure

Weights

- **Midterm** = $0.6 \cdot \text{RAP1} + 0.4 \cdot \text{Quiz-1}$ · **Final** = $0.7 \cdot \text{RAP2} + 0.3 \cdot \text{Quiz-2}$
- **Course grade** = $0.4 \cdot \text{Midterm} + 0.6 \cdot \text{Final}$
- **RAP1 (Week 7)**: first half of the product (S2–S5 + basic protections). **RAP2 (Week 15)**: full product + S16 results.

Learning outcomes (LO)

LO	Definition	Bloom level	Knowledge/Skill	RAP1	RAP2
LO.1	Identifies and classifies common software vulnerabilities (buffer overflow, injection attacks, memory leaks, etc.).	Understand	KNOWLEDGE	✓	✓
LO.2	Explains basic encryption methods and secure communication principles (SSL/TLS) for protecting sensitive data.	Understand	KNOWLEDGE	✓	✓
LO.3	Explains code hardening techniques (input validation, secure memory management, RASP, code obfuscation) and applies them.	Apply	SKILL	✓	✓
LO.4	Explains the principles of building secure communication channels using encryption and authentication mechanisms.	Apply	SKILL	✓	✓
LO.5	Creates a protection plan using secure software design principles (least privilege, defence in depth, etc.).	Synthesize	SKILL	✓	✓
LO.6	Performs a basic security review and vulnerability assessment to detect software vulnerabilities.	Analyze	SKILL	✓	✓
LO.7	Knows secure programming standards (e.g. ETSI, EMV, FIPS) and the principles of penetration test planning.	Understand	KNOWLEDGE	✓	✓

Both checkpoints touch every learning outcome; RAP1 measures the basic/early level, RAP2 the advanced level and the full product.

3. Tools and setup

Tool	Purpose	Output	Delivery condition
C++ compiler (GCC/Clang/MSVC)	Building the console app + DLL/.so	Working binary	Builds without errors in release
CMake + CTest	Build system + unit testing	Test report	Green in CI
SQLite	Data-at-rest store	.db schema	Records encrypted with AES-GCM
SoftHSM / PKCS#11	Key-wrapping/storage simulation	Key-store configuration	Critical keys stay wrapped
OpenSSL	Cryptography, TLS, signing	Library linkage + certificates	Correct algorithm/mode/padding
Doxygen	Source documentation	PDF/HTML output	Delivered in the template's format
Git / GitHub	Version control, collaboration	Commit history, pull request	Private repository, both members' commits visible
GitHub Actions	Continuous integration (CI)	Green build/test badge	CI green before merging

Template repository: [ucoruh/cpp-cmake-ctest-template](#)

Do not fork. A fork of a public repository cannot be made private on GitHub, so create your own repository from the template instead: on the template page click **Use this template** → **Create a new repository**, choose the owner, type the repository name `cen429-project-name-surname-cpp`, select **Private** and create it. Then add the instructor (`ucoruh`) and your teammate (if any) as **collaborators** (*Settings* → *Collaborators*).

Use the template fully

The template provides build, **CTest unit testing**, documentation generation, test/documentation coverage, and packaging. Deliverables that do not follow the template are not accepted. Produce releases; record the version id and digests (Week 12 TOE identity).

Project layout:

- `lib/` — security library (DLL/.so): cryptography, RASP, code obfuscation, data-protection functions.
- `app/` — console application: the user interface that uses the library.
- `test/` — CTest unit tests (crypto and protection functions).

For environment setup and background knowledge, see [Prerequisites](#).

Showing your project locally

A private repository on GitHub Free has no GitHub Pages site, so you show everything **on your own computer**:

1. Run `7-build-all-windows.bat` (or `7-build-all-linux.sh` on Linux/WSL). It builds the project, runs the tests and produces every report.
2. Run `9-open-site-windows.bat` (or `9-open-site-linux.sh`). The full project site opens on `http://localhost`: every report (tests, code coverage, documentation coverage, separately for Windows and Linux) and the API docs.
3. The `release/` folder holds every output: the app/exe, the library, every report, the API docs, `site.zip`, the source, `ASSETS.md` and `SHA256SUMS.txt`.
4. Make a GitHub Release with `10-release-windows.bat` (or `10-release-linux.sh`). Releases work on private repositories, and collaborators can see them.

Each template's README and guide have the details; see the page "Showing your project without GitHub Pages" in the template's `docs/guide/` folder: [C/C++](#).

Demo checklist

Demo slots. Each team has a **10-minute slot** (about 2 minutes per member) during the class hours of the demo day. After team selection closes, a slot list is published on Microsoft Teams; come to your slot with the project already built and the local site open. Every member answers at least one question.

What you open and show, in this order:

1. The **home page** of the local site (`9-open-site-...`, on `http://localhost`).
2. **Each report page**: tests, code coverage, documentation coverage (Windows and Linux).
3. The **API docs**.
4. The `release/` folder listing.
5. **Run the app** from the release archive.
6. The **GitHub Release page** of your private repository (the instructor is a collaborator).

4. Topic selection

Project topics are listed in the **Appendix – Project topic list** at the end of this page (100 topics). Each topic box gives a short description of the application, its key features, the **assets to protect**, and the **security requirements that stand out** for that topic (using the items in Section 5).

How to choose

1. Browse the list at the end of the page and choose one topic.
2. Record your choice in the shared **team and topic list** (link in the Microsoft Teams post). **Each topic can be taken by only one team**; the first team to record it gets it.
3. Have it approved by the instructor together with your project plan (prepared alongside Section 3, Tools and setup); once approved, the topic cannot be changed.

- **All 15 requirements** in Section 5 apply to every project; the "highlighted requirements" in a topic box show where they fit most naturally and earn the most credit for that topic.

- If you have an idea that is not on the list, you may choose it with the instructor's approval, provided it is rich enough to apply the requirements meaningfully.
- If you are retaking this course, choose a topic **different from your previous project**.

5. Requirements

Each requirement notes **which week it is taught** and **which guide section (S)** it is documented in. The 15 requirements below are grouped into midterm and final scope to match the rubric (see Section 8, Rubrics); the advanced aspects of some requirements deepen in later weeks. Meet each item, or **defer/scope it out with justification** (Week 13, S14).

5.1 Midterm scope (RAP1)

4.1 Development environment security · (Weeks 1, 12 · S13)

Related weeks: [Week 1](#) · [Week 12](#)

- Software development flow and **change management** (baseline → request → classify → approve → release → verify).
- Version control with **Git**; access logging; signed releases.
- Development host/server security (short policy).

4.2 Data-in-use security · (Weeks 3, 6 · S7)

Related weeks: [Week 3](#) · [Week 6](#)

- Sensitive data in memory is **wiped securely** after use (`memset_s`-like).
- Runtime data protection (shadow copy / integrity — Week 6).

4.3 Data-in-transit security · (Weeks 3, 10 · S6, S11)

Related weeks: [Week 3](#) · [Week 10](#)

- **TLS 1.3**; certificate chain + **SAN** validation; **pinning** + backup pin.
- Encrypted **session key**; **device/version binding**; integrity + authentication; server verification code.

4.4 Data-at-rest security · (Weeks 3, 10, 11 · S8)

Related weeks: [Week 3](#) · [Week 10](#) · [Week 11](#)

- File/DB encryption with **AEAD** (AES-GCM); non-repeating nonce.
- For sensitive keys, **whitebox** or **SoftHSM/PKCS#11**; **justify** the choice (Week 11).

4.8 Interface definitions and protection · (Weeks 1, 4 · S3, S6)

Related weeks: [Week 1](#) · [Week 4](#)

- All interfaces protected with access control + authentication; input validated at the **trust boundary**.

4.9 Code hardening · (Weeks 4, 9, 14 · S9)

Related weeks: [Week 4](#) · [Week 9](#) · [Week 14](#)

- Opaque loops/predicates, name/file/string/arithmetic obfuscation, opaque booleans, bogus ops/dead branches, **control-flow flattening** + random exit, logging **off** in release. **Measure** the cost (Week 9/14 demos).

4.10 RASP · (Week 6 · S10)

Related weeks: [Week 6](#)

- Checksum integrity, caller hash/signature verification, root/emulator detection, **hook/anti-debug**, tamper detection + response, control-flow counter.

4.11 Memory protection · (Week 4 · S9)

Related weeks: [Week 4](#)

- Compiler/OS protections (stack canary, PIE, RELRO, NX, CFI); sensitive memory cleared after use.

5.2 Final scope (RAP2)

4.5 Static asset protection · (Weeks 1, 4, 9 · S5, S9)

Related weeks: [Week 1](#) · [Week 4](#) · [Week 9](#)

- Secret keys, digests, source code, resources: encryption + access control + obfuscation.

4.6 Dynamic asset protection · (Weeks 3, 6 · S5, S8)

Related weeks: [Week 3](#) · [Week 6](#)

- Device/app fingerprints, session data, dynamic keys are encrypted.

4.7 Asset management · (Weeks 1, 3, 13 · S5)

Related weeks: [Week 1](#) · [Week 3](#) · [Week 13](#)

Per asset: **name**, **description**, **location** (table/column), **source**, **size**, **creation/destruction time**, **default**, **protection scheme** (C/I/I+).

4.12 Cryptography and certificates · (Weeks 3, 10 · S8, S11)

Related weeks: [Week 3](#) · [Week 10](#)

- Correct algorithm/mode/padding; signature verification (`==1` , version coverage); SSL/TLS + pinning + mutual authentication.

4.13 Certification and penetration test plan · (Weeks 12, 13 · S16, S14, S17)

Related weeks: [Week 12](#) · [Week 13](#)

- Standards mapping (ETSI/EMVCo/GSMA/PCI/MASVS); **penetration test plan** (scope, rules, methodology, test card) and its **results**; attack potential + CVSS.

4.14 Binary application protection · (Weeks 6, 9, 11, 14 · S9, S10, S15)

Related weeks: [Week 6](#) · [Week 9](#) · [Week 11](#) · [Week 14](#)

- **Detection** (checksum, anti-debug, emulator), **defence** (obfuscation, string/resource encryption, call hiding), **deterrence** (response/shutdown policy).

4.15 OWASP and build/deploy pipeline · (Weeks 5, 14 · S13, S15)

Related weeks: [Week 5](#) · [Week 14](#)

- OWASP MASVS/ASVS principles; **SBOM** (CycloneDX) + dependency scan; build pipeline with obfuscation + signing.

5.3 Rules for both checkpoints

- All values must be **synthetic**; no real secret, key, or personal data may be in the repository.
- Every requirement item must be **visible** in the code and **documented** in the matching S section of the security guide; an undocumented implementation counts as unmet.
- If a requirement cannot be met, record the **justification** in S14 (assumptions and deferrals); it cannot be silently skipped.
- The application runs only on your own computer (localhost); it must not harm any other system or data.

6. Deliverables

6.1 Deliverables table

#	Deliverable	Format	Rule
1	Source code	Git repository (created from the template)	CMake/CTest, C++ DLL/.so + SQLite + SoftHSM integration; both members' commits visible
2	Unit tests + CI record	CTest report + CI log	Crypto and protection functions are tested; no merge without green CI
3	Security guide (S0–S17)	.docx / .pdf	Midterm: S0–S5 draft · Final: S0–S17 complete, a scaled-down model of a certification document
4	Test results (S16)	Guide section	Final only; not a plan, the observed results
5	Compliance matrix (S17) + deferred requirements (S14)	Guide section	Final only; every deferred item is justified
6	SBOM + dependency scan	CycloneDX file	Final only; delivered together with 4.15
7	Release	Git tag	Midterm: <code>midterm-v1.0</code> · Final: <code>final-v1.0</code>
8	Demo	Live presentation (10 min, slot list after team selection)	Midterm: Week 7 · Final: Week 15

6.2 Security guide sections (S0–S17)

No	Section	No	Section
S0	Cover, version history	S9	Code hardening (advanced)
S1	Scope, abbreviations, references	S10	RASP + response policy
S2	Product and architecture	S11	Secure communication (TLS)
S3	Interfaces	S12	Reporting/logging
S4	Threat model (STRIDE, tree)	S13	Dev process, SBOM, change mgmt
S5	Asset list + protection	S14	Assumptions and deferrals
S6	Identity/binding	S15	Build, signing, deploy pipeline
S7	Data security + shell matrix	S16	Security testing and results
S8	Crypto, key lifecycle	S17	Requirement compliance matrix

6.3 Single ZIP archive structure

```

cen429-midterm-name-surname.zip
├── cen429-project-name-surname-cpp/
│   ├── lib/
│   ├── app/
│   ├── test/
│   ├── docs/
│   ├── security-guide/
│   │   └── cen429-security-guide-name-surname.docx
│   ├── sbom/
│   ├── test-coverage/
│   └── README.md
└── # final delivery: cen429-final-name-surname.zip
    # clone of the GitHub repo (gitignore applied)
    # security library (DLL/.so)
    # console application
    # CTest unit tests
    # Doxygen output
    # S0–S17 security guide
    # CycloneDX SBOM + dependency scan (final)
    # CTest coverage report

```

Naming: repository `cen429-project-name-surname-cpp`; archive `cen429-midterm-name-surname.zip` / `cen429-final-name-surname.zip`; security guide file `cen429-security-guide-name-surname.docx`. The cover page of the report/guide must include the **GitHub repository link**.

7. Team workflow and engineering practices

- **GitHub Flow:** `main` is protected and never committed to directly; each task gets its own feature branch, merged through a **pull request**.
- **Commit message convention:** [Conventional Commits](#), e.g. `feat(crypto): add AES-GCM wrapper`, `fix(rasp): correct debugger detection`, `test(hash): add SHA-256 vectors`.
- **Pull request + review:** every PR is **reviewed** by at least one teammate; the description states what changed and how it was tested.

- **Issues + Projects board:** every task is a GitHub Issue; the board is tracked through Backlog → In Progress → In Review → Done.
- **No merge without green CI:** GitHub Actions runs the build and CTest; a red CI blocks merging into `main`.
- **Version tags:** the midterm delivery is tagged `midterm-v1.0`, the final delivery `final-v1.0` (release).
- **Definition of Done:** the code builds and runs · new public functions have unit tests · coverage did not drop · the PR was reviewed and approved · CI is green · the matching S section of the security guide was updated.

8. Rubrics

Each criterion is scored on a **1–5** scale: points = (level ÷ 5) × criterion points.

8.1 Achievement levels

Level	Meaning
5 – Excellent	Everything in scope works, is tested and documented; can be explained step by step in the demo
4 – Good	Minor gaps or edge-case bugs; complete and tested overall
3 – Adequate	Basic operations work; clear gaps in tests, documentation or measurements
2 – Poor	Compiles, but most operations are wrong or missing; weak explanation
1 – No evidence	Not submitted or not working

8.2 RAP1 – Midterm checkpoint rubric (Week 7, 100 points)

#	Criterion	Scope	LO	Points
1	Threat model and asset list	S4, S5	LO.1, LO.5	15
2	Data security: in transit, at rest, in use	4.2–4.4; S7, S8	LO.2, LO.4	20
3	Code hardening and memory protection	4.9, 4.11; S9	LO.3	20
4	RASP and response policy	4.10; S10	LO.3	10
5	Development environment and change management	4.1; S13	LO.5	10
6	Unit tests and CI	—	LO.6	10
7	Security guide (S0–S5) and demo	—	LO.7	15
Total				100

8.3 RAP2 – Final checkpoint rubric (Week 15, 100 points)

#	Criterion	Scope	LO	Points
1	Cryptography and certificates	4.12; S8, S11	LO.2, LO.4	15
2	Binary application protection, obfuscation and diversification	4.14; S9, S10, S15	LO.3	20
3	Asset management and dynamic assets	4.5–4.7; S5	LO.5	10
4	Security testing and observed results	4.13; S16	LO.6	20
5	Standards, compliance matrix, deferrals	S1, S14, S17	LO.7	15
6	OWASP, SBOM and build/deploy pipeline	4.15; S13, S15	LO.1, LO.5	10
7	Report, presentation and demo	—	LO.7	10
Total				100

Every row needs **evidence**: a "met" row without evidence counts as unmet (Week 13).

9. Acceptance conditions

⚠ Submissions are not accepted if...

- there is no GitHub repository, it is not **private**, or team members have no commits,
- the proportion of requirements met is below the minimum threshold,
- the repository or archive contains **binary files** (compiled `.exe` / `.dll` / `.so`),
- no release has been produced,
- the application does not **build/run** on Windows or WSL/Linux,
- the repository contains a real secret, key, or personal data (all values must be synthetic),
- **plagiarism** is detected.

10. Questions asked in the demo

- **Git/GitHub:** Did you create the repository from the template with the correct name? Is the repository private, is the instructor a collaborator? Do both members have commits? Were branches used, how were merges resolved?
- **Setup and build:** Build the application and the library on Windows and in WSL/Linux; show the `lib` / `app` / `test` split and their dependencies.
- **Topic (line by line):** Explain your chosen security requirement (e.g. AES-GCM encryption, RASP tamper check, a code-hardening transform) line by line in the code; draw the memory/data flow.
- **Tests and documentation:** Open the unit tests and coverage report; show the matching S section of the security guide.
- **File/data operations:** Add a record, close and reopen the program, show that the data comes back encrypted.
- **Programming:** Memory management, pointers, compiler/OS protections (stack canary, ASLR, NX, CFI), inspecting variables and the stack in a debugger.

11. Professional responsibility and academic integrity

- You must be able to **explain** every line you submit; any team member may be questioned during the demo.
- Cite the source if you reuse someone else's code or text; watch for license compatibility.
- All examples and values must be **synthetic**; no real personal data may be used.
- **Plagiarism** and unauthorized copying cause failure; similarity checks may be run.
- Individual contribution is tracked from the commit history; committing on someone else's behalf (a ghost commit) counts as plagiarism.

⚠ Safe and legal frame

Penetration testing is only done on **your own** project. Examples and values must be **synthetic**; no real secret, key, or personal data may enter the repo. Your application must not harm anyone else's system or data.

12. Frequently asked questions

? Can I form a team by myself?



Yes. The team is at most 5 students; you may also work alone. Teams are fixed at the end of week 3 (04.10.2026) and cannot change afterwards. The requirements and rubric do not change.

? Can I change my topic after it is approved?



No. Once a topic is approved together with the project plan, it cannot be changed. If you need a change, discuss it with the instructor.

? What happens if I cannot fully meet a requirement?



You may defer it or scope it out by recording your justification in S14 (assumptions and deferrals) of the security guide (Week 13). An item silently skipped counts as unmet.

? Can a requirement left incomplete at RAP1 be finished at RAP2?



Yes, the final scope covers the full product and gaps can be completed; however, the RAP1 grade reflects the state at the time of that demo and is not updated afterwards.

? In what format do we submit the security guide?



As a `.docx` or `.pdf`, structured like a scaled-down certification document, with sections S0–S17. A draft of S0–S5 is expected at midterm, and the complete S0–S17 at final.

? Are late submissions accepted?



The rules in the syllabus apply: late submissions are not accepted. Notify the instructor as soon as possible if something unexpected happens.

? Where can I find the detailed scoring key?



Section 8 on this page gives the criteria, scope and points; additional evidence templates and clarifications may be shared on Microsoft Teams if needed.

Appendix – Project topic list

Choose one of the topics below (see 4. Topic selection). The topics are in four groups: **payments and commerce (01–25)**, **health, education, public services and identity (26–50)**, **media, licensing, communication and the software supply chain (51–75)**, **IoT, industry, transport and the enterprise (76–100)**. All data is synthetic; the server side is a simulation running only on your own computer (localhost).

01 – Offline Payment Wallet



Summary: A wallet that lets a user make small payments without an internet connection. The balance and single-use payment keys live on the device; transactions reconcile with the server once connectivity returns. The device owner is also the likely attacker: they want to inflate the balance or spend the same key twice.

Key features: Account creation and PIN · Balance display · Offline payment (QR/code) · Transaction history · Server reconciliation

Assets to protect:

- Single-use payment keys — **C, I+**
- Balance and transaction counter — **I+** (must not roll back)
- PIN verification data (hash) — **C, I**
- Session key — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** balance and keys in SQLite with AES-GCM; the key is device-bound and wrapped in SoftHSM.
- **4.3 Data-in-transit security:** reconciliation over TLS 1.3 with certificate pinning; every transaction is MAC'd and counter-bound (no replay).
- **4.10 RASP:** keys are wiped and the payment refused when a debugger or tampering is detected.
- **4.11 Memory protection:** the PIN and key are securely erased after use.

Server (localhost simulation): Flags spent keys and rejects any counter rollback.

Extension: Direct device-to-device payment; a second approval step when a limit is exceeded.

02 — Personal Password Vault



Summary: A desktop application where a user stores passwords for different accounts in a local vault unlocked by one master password. The vault file can be stolen, or someone with physical access to the device (a family member, a thief, malware) may try to guess the master password or capture it in memory.

Key features: Vault creation and master password setup · Add/edit account-password entries · Random strong password generator · Clipboard copy with auto-clear · Vault lock/auto-lock · Backup

Assets to protect:

- Key derived from the master password — **C**
- Account passwords stored in the vault — **C, I**
- Vault file integrity tag — **I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** the vault file is encrypted with AES-256-GCM; the key is derived from the master password with Argon2id.
- **4.11 Memory protection:** unlocked passwords are securely wiped from memory after use and never written to the swap file.
- **4.12 Cryptography and certificates:** key-derivation parameters (salt, iterations) are documented and follow current recommendations.
- **4.2 Data-in-use security:** a password copied to the clipboard is cleared automatically after a short delay.

Server (localhost simulation): Simulates only the sync of the encrypted backup file; it cannot decrypt any password.

Extension: Two-factor unlock (hardware-key simulation) and vault sharing (family/team).

03 — One-Time Password (TOTP) Authenticator



Summary: An application that generates short-lived, six-digit codes as a second authentication factor for a user's accounts. An attacker who compromises the device or copies the seed can sign into the victim's accounts without the second factor.

Key features: Add an account via QR/text (otpauth) · 30-second code generation with countdown · Managing multiple accounts · Backup recovery codes · Secure transfer on device change · Account removal

Assets to protect:

- TOTP seed values — **C, I+**
- Device-binding key — **C**
- Backup recovery codes — **C, I**

Highlighted requirements:

- **4.4 Data-at-rest security:** seeds are stored encrypted in SQLite, with the key wrapped in SoftHSM.
- **4.6 Dynamic asset protection:** a seed only works on the enrolled device; export is explicitly flagged.
- **4.12 Cryptography and certificates:** RFC 6238-compliant HMAC-SHA1/256 code generation, with documented time-skew tolerance.
- **4.3 Data-in-transit security:** any seed transfer during enrolment (if present) is protected with TLS.

Server (localhost simulation): Simulates seed issuance/initial pairing at enrolment; code verification stays client-side.

Extension: Push-based approval (yes/no) added as a second method.

04 — Software POS Simulator



Summary: A desktop point-of-sale application that records a small shop's card/cash sales. A cashier or an intruder may try to alter receipts, delete logs, or create fraudulent refunds.

Key features: Product/basket management · Completing a sale and issuing a receipt · Refunds (access-controlled) · Daily sales summary · Cashier sign-in · Server synchronisation

Assets to protect:

- Transaction records and receipts — **I+**
- Terminal signing key — **C, I+**
- Cashier authentication data — **C, I**

Highlighted requirements:

- **4.7 Asset management:** secure injection of the terminal key at setup and its periodic rotation are planned.
- **4.3 Data-in-transit security:** the sales summary is sent to the server over TLS with mutual authentication.
- **4.14 Binary application protection:** the executable is signed against tampering and its integrity is checked at start-up.
- **4.13 Certification and penetration test plan:** a test plan covers fraudulent-refund and receipt-tampering scenarios.

Server (localhost simulation): Aggregates end-of-day totals; it does not accept a record without a verified terminal identity and signature.

Extension: A multi-terminal store scenario with central stock synchronisation.

05 — Cold Crypto Wallet Simulator



Summary: A simulation in which the private key never leaves an offline device: transactions are signed offline and carried to a network-connected component via a QR code or file. Malware, or anyone with physical access, wants to exfiltrate the private key.

Key features: Key-pair generation and recovery-phrase display · Offline transaction signing · Exporting a signed transaction (QR/file) · Balance computation (using data from the online component) · Restoring a wallet from its recovery phrase

Assets to protect:

- Private key / recovery phrase — **C, I+**
- Signed transactions — **I+**
- Wallet file — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** the private key is AES-GCM encrypted on disk, wrapped in SoftHSM.
- **4.12 Cryptography and certificates:** ECDSA/Ed25519 signing, with an approved entropy source for recovery-phrase generation.
- **4.14 Binary application protection:** the library holding the signing logic is hardened against reverse engineering.
- **4.9 Code hardening:** key-derivation and signing symbols are obfuscated.

Server (localhost simulation): Plays only the online component simulating balance/transaction history; it never sees the private key.

Extension: A multi-signature (m-of-n) approval flow.

06 — Loyalty Points Wallet



Summary: A wallet application where a customer accumulates points from purchases and spends them in stores. The user may try to alter the local balance to mint points, or spend the same points at several stores.

Key features: Earning points (receipt scan/code entry) · Balance display · Spending points in-store (QR/code) · Transaction history · Campaign notifications · Server synchronisation

Assets to protect:

- Points balance — **I+**
- Spend transaction code — **C, I**
- Sync session key — **C**

Highlighted requirements:

- **4.6 Dynamic asset protection:** the local balance is display-only; the real balance is verified against the server on every use.
- **4.8 Interface definitions and protection:** the earn/spend API is clearly defined and requires a signed request.
- **4.3 Data-in-transit security:** synchronisation is protected with TLS 1.3.
- **4.2 Data-in-use security:** the spend-time code is short-lived and shown on screen for a limited time only.

Server (localhost simulation): Holds the authoritative balance; it never trusts a balance the client reports, only its own record.

Extension: Cross-store point transfer and gifting.

07 – Prepaid Transit Card Simulator



Summary: A simulation, capable of working offline, in which value loaded onto a transit card is deducted at the turnstile. The card/device owner may try to duplicate (clone) the balance or roll back the ride counter.

Key features: Loading value (when online) · Simulated turnstile pass (balance deduction) · Transaction/pass history · Card status lookup · Card blocking (lost/stolen) · Server reconciliation

Assets to protect:

- Card balance and pass counter – **I+** (must not roll back)
- Top-up integrity code (MAC) – **C, I**
- Card identifier – **I**

Highlighted requirements:

- **4.4 Data-at-rest security:** the balance and counter are kept encrypted and MAC'd in local storage.
- **4.6 Dynamic asset protection:** the counter only moves forward; rollback or replay is rejected.
- **4.12 Cryptography and certificates:** top-ups are MAC'd with a symmetric key bound to the card/device.
- **4.9 Code hardening:** the balance-update logic is hardened to make cloning difficult.

Server (localhost simulation): Approves top-ups and, during periodic reconciliation, compares the card's history against its own record.

Extension: A family card – managing several cards from one account.

08 — Expense Receipt Vault



Summary: An application in which employees photograph and store expense receipts as evidence for an approval process. The user, or someone intercepting the flow, may try to alter the receipt image or the amount after approval.

Key features: Adding a receipt photo · Tagging amount/category · Submitting for approval · Approval status tracking · Searching the receipt archive · Exporting a report

Assets to protect:

- Receipt image files — **C, I+**
- Receipt metadata (amount, date) — **I+**
- Approval signature — **I+**

Highlighted requirements:

- **4.5 Static asset protection:** receipt images are stored encrypted on disk and cannot be read directly outside the application.
- **4.12 Cryptography and certificates:** the content is hashed and signed on submission; any later change is detected.
- **4.7 Asset management:** the retention period and archive-deletion policy for receipts are defined.
- **4.13 Certification and penetration test plan:** receipt-forgery and amount-tampering scenarios are added to the test plan.

Server (localhost simulation): Keeps the hash/signature record of approved receipts and flags any inconsistency when a later receipt is compared against it.

Extension: A multi-approver workflow (manager + finance).

09 – Encrypted Personal Budget Ledger



Summary: A local application where a user records income and expenses by category. Anyone who gains access to the device may want to read sensitive personal information such as spending habits.

Key features: Adding income/expense entries · Defining categories and budget limits · Monthly summary and chart · Exporting an encrypted backup · Locking with a master password · Multiple accounts/wallets

Assets to protect:

- Income/expense entries – **C**
- Budget limits and categories – **I**
- Key derived from the master password – **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** entries are kept AES-GCM encrypted in SQLite.
- **4.2 Data-in-use security:** entries are decrypted in memory only while unlocked; visibility ends on lock.
- **4.11 Memory protection:** the master password and derived key are wiped from memory after use.
- **4.1 Development environment security:** test data never contains real financial data; only synthetic data is used.

Server (localhost simulation): Simulates only the optional upload of the encrypted backup file; it cannot decrypt its contents.

Extension: Shared, permissioned categories across family members.

10 – 🎁 Gift Card Issuing and Validation System



Summary: A system where a store issues gift-card codes and the till validates a code and deducts its balance. An attacker may try to guess a valid code or redeem the same code more than once.

Key features: Issuing a card code (fixed amount) · Validating a code and checking its balance · Partial redemption (remaining balance) · Cancelling a card · Transaction log · Batch card issuance

Assets to protect:

- Card codes and their balances – **C, I+**
- Code-signing/HMAC key – **C, I+**
- Transaction log – **I**

Highlighted requirements:

- **4.12 Cryptography and certificates:** card codes are randomly generated and HMAC-verified, making guessing infeasible.
- **4.8 Interface definitions and protection:** the validation interface is rate-limited against brute forcing.
- **4.6 Dynamic asset protection:** a code can be fully redeemed only once; the race condition is prevented.
- **4.13 Certification and penetration test plan:** a test plan is prepared for code guessing and double-redemption scenarios.

Server (localhost simulation): Holds the single authoritative record of card codes and remaining balance, queuing concurrent redemption requests.

Extension: Restricting a card to a specific store chain (whitelisting).

11 – Microcredit Application Client



Summary: A client where a user enters and submits personal and financial information for a small loan application. The application data is sensitive to the user and, if altered by an intercepting party, can change the loan decision.

Key features: Identity and income form · Attaching documents (synthetic income proof) · Submitting the application · Checking application status · Past applications · Notifications

Assets to protect:

- Identity and financial information – **C, I**
- Application documents – **C, I+**
- Session/request signature – **I**

Highlighted requirements:

- **4.2 Data-in-use security:** sensitive fields are kept in cleartext in memory only for as long as the form requires.
- **4.4 Data-at-rest security:** the draft application is stored encrypted on the local disk.
- **4.3 Data-in-transit security:** submission is protected with TLS 1.3, with server certificate verification.
- **4.15 OWASP and build/deploy pipeline:** inputs are also validated server-side; injection-class flaws are scanned for in the build pipeline.

Server (localhost simulation): Receives applications and performs a synthetic pre-screening score; it does not make a real credit decision.

Extension: A document-verification step (synthetic signature check) is added.

12 — \$ Currency Exchange Till



Summary: A till application where a currency exchange office buys and sells at current rates. A cashier or an outsider may try to alter a rate value for unfair gain.

Key features: Displaying the current rate list · Computing a buy/sell transaction · Tracking till balances by currency · Generating a transaction slip · End-of-day reconciliation · Rate update (authorised)

Assets to protect:

- Rate list — I+
- Till balances — I+
- Transaction records — I

Highlighted requirements:

- **4.6 Dynamic asset protection:** the rate list can only change through an authorised channel; value consistency is checked during a transaction.
- **4.7 Asset management:** periodic retrieval of the current rate from a trusted source and its cache validity period are managed.
- **4.12 Cryptography and certificates:** rate updates are signed by the server; the client never applies an unsigned update.
- **4.9 Code hardening:** the calculation logic is hardened against in-memory or binary-level alteration.

Server (localhost simulation): Publishes a signed rate list; till applications accept only this signed list.

Extension: A multi-branch office scenario with central limit control.

13 – Children's Allowance Wallet



Summary: A wallet where a parent sets a child's digital allowance and remotely manages a spending limit. A child with physical access to the device may try to alter the spending limit or balance in their own favour.

Key features: Parent-side balance/limit setup · Child-side balance display and spending · Spending category restrictions · Transaction history (visible to the parent) · Parent-child device pairing · Notifications

Assets to protect:

- Balance and spending limit – **I+**
- Parent-child pairing key – **C, I**
- Transaction history – **I**

Highlighted requirements:

- **4.4 Data-at-rest security:** the balance and limit on the child's device are kept encrypted and integrity-protected.
- **4.6 Dynamic asset protection:** the limit and balance can change only via a signed command from the parent's device; local edits are rejected.
- **4.8 Interface definitions and protection:** the parent-child pairing interface requires mutual authentication.
- **4.9 Code hardening:** the balance/limit control logic is hardened against local tampering.

Server (localhost simulation): Verifies the parent-child pairing and command signatures, and centrally logs balance changes.

Extension: A single parent dashboard covering several children/accounts.

14 — Sealed-Bid Auction Client



Summary: An auction client that keeps every participant's bid secret from everyone, including the server, until a predetermined reveal moment. A bid disclosed early or altered afterwards grants an unfair advantage.

Key features: Creating and submitting a bid (commitment) · Own bid history · Revealing the bid at opening time · Determining the winner · Viewing auction status

Assets to protect:

- Bid amount (until reveal) — **C**
- Commitment value — **I+**
- Reveal key/salt — **C, I**

Highlighted requirements:

- **4.2 Data-in-use security:** the bid amount is only assembled in memory at submission time and is not kept on screen unnecessarily.
- **4.12 Cryptography and certificates:** the bid is first submitted as a hash commitment and verified with the salt at reveal time.
- **4.3 Data-in-transit security:** the commitment and reveal data are protected with TLS.
- **4.13 Certification and penetration test plan:** early bid-leakage and post-commitment tampering scenarios are tested.

Server (localhost simulation): Collects commitments and cannot resolve any of them until reveal time; after reveal it compares all bids and announces the winner.

Extension: A minimum-increment rule with automatic time extension on a late bid.

15 — E-Receipt Signer



Summary: A component that digitally signs an electronic receipt issued at a point of sale, giving the customer verifiable proof. An attacker may want to forge a receipt and pass it off as genuine.

Key features: Building the receipt content · Digital signing · Signature verification (customer/auditor side) · Signed receipt archive · QR-based verification link · Key/certificate display

Assets to protect:

- Signing private key — **C, I+**
- Signed receipt records — **I+**
- Certificate chain — **I**

Highlighted requirements:

- **4.12 Cryptography and certificates:** signing with RSA/ECDSA, with certificate-chain and expiry checks applied.
- **4.7 Asset management:** generation, storage, and renewal of the signing key at expiry are planned.
- **4.4 Data-at-rest security:** the private key is kept wrapped via [SoftHSM/PKCS#11](#); the application never sees the raw key.
- **4.14 Binary application protection:** the binary that invokes signing is protected so the key cannot be exported directly.

Server (localhost simulation): Stores the hash of signed receipts and confirms a receipt's authenticity when a QR-based verification request arrives.

Extension: Batch receipt signing and timestamp integration.

16 — Insurance Claim Client

Summary: A client where a policyholder builds a claim with damage photos and a description. The claimant, or an intercepting party, may try to alter the damage photo or the amount after submission to obtain an unfair payout.

Key features: Adding a damage description and photo · Entering policy details (synthetic) · Submitting the claim · Status tracking · Uploading supporting documents · Notifications

Assets to protect:

- Damage photos and description — **C, I+**
- Policy/identity information — **C**
- Application integrity digest — **I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** the draft claim and photos are kept encrypted locally.
- **4.5 Static asset protection:** photo files are protected from direct access outside the application.
- **4.12 Cryptography and certificates:** content is hashed at submission time; any later change is detected by comparing the digest on every subsequent access.
- **4.15 OWASP and build/deploy pipeline:** the upload path is hardened against common upload flaws through type/length validation.

Server (localhost simulation): Preserves the integrity of the submitted claim via its digest; any later "update" request is processed as a new record.

Extension: Cross-checking by two independent assessors (simulated).

17 — 📄 Donation Tracker with Signed Receipts



Summary: A tracking application in which a donor's contributions are recorded with digitally signed receipts. A bad actor may want to forge a receipt or redeem the same receipt more than once.

Key features: Recording a donation · Generating and signing a receipt · Verifying a receipt · Annual donation summary · Searching donation history · Exporting a report

Assets to protect:

- Receipt-signing key — **C, I+**
- Receipt records — **I+**
- Donor information (synthetic) — **C**

Highlighted requirements:

- **4.12 Cryptography and certificates:** every receipt is signed with a unique serial number.
- **4.6 Dynamic asset protection:** reuse of a receipt number (presenting the same receipt twice) is rejected server-side.
- **4.7 Asset management:** long-term retention of the receipt archive and its access authorisation are planned.
- **4.13 Certification and penetration test plan:** a test plan targets receipt forgery and reuse scenarios.

Server (localhost simulation): Keeps the status of every issued receipt number and checks, on verification, whether it has already been used.

Extension: Batch receipt generation for corporate donations.

18 – Encrypted Subscription and Payment Reminder



Summary: An application that tracks a user's subscription payments and the last four masked digits of a card, sending reminders. Anyone with device access may want to see spending habits and subscription data, even in masked form.

Key features: Adding a subscription (name, amount, date) · Recording a masked payment method · Upcoming payment reminders · Monthly total summary · Marking a subscription cancelled · Encrypted backup

Assets to protect:

- Subscription and amount records – **C**
- Masked payment metadata – **C**
- Key derived from the master password – **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** all records are kept AES-GCM encrypted in SQLite.
- **4.2 Data-in-use security:** card data is handled so that only the last four digits are ever shown; the full number is never retained.
- **4.1 Development environment security:** only synthetic data is used during development and testing; real data never enters the codebase.
- **4.11 Memory protection:** the unlocking master password is cleared from memory after use.

Server (localhost simulation): Simulates only the optional retrieval of the encrypted backup.

Extension: Shared subscription tracking among family members.

19 — End-of-Day Till (Z) Reporter



Summary: An application in which a cashier closes the till at day's end, producing an immutable "Z report". A cashier or manager may try to alter the report in their own favour after closing.

Key features: Viewing the day's sales totals · Entering the cash count · Generating the Z report (closing) · Report archive and search · Computing the variance (till shortage/overage) · Exporting the report

Assets to protect:

- Z report (post-closing) — **I+**
- Cash-count data — **I**
- Report-signing key — **C, I+**

Highlighted requirements:

- **4.6 Dynamic asset protection:** once a Z report is closed, no field can be edited; a correction requires a new record.
- **4.12 Cryptography and certificates:** the report is hashed and signed at closing; the signature is stored alongside the report.
- **4.9 Code hardening:** the closing and signing flow is hardened against direct edits in application memory.
- **4.13 Certification and penetration test plan:** test scenarios cover attempts to alter a closed report.

Server (localhost simulation): Stores the signed digest of every closed Z report and rejects a second closing for the same day.

Extension: Shift-level interim reports (X reports) are added.

20 – Corporate Expense Card Limit Manager



Summary: An application that manages the spending limits on virtual expense cards a company issues to employees. An employee may try to raise their own card's limit or category without authorisation.

Key features: Defining a card (limit, category) · Creating a spend request · Manager approval workflow · Spending history · Updating limit/category (authorised) · Suspending a card

Assets to protect:

- Card limit and category rules – I+
- Masked card metadata – C
- Approval authority/role – I

Highlighted requirements:

- **4.8 Interface definitions and protection:** the limit-update interface is open only to the manager role, verified server-side.
- **4.6 Dynamic asset protection:** an employee's client cannot alter its own limit locally; every request is approved against the current server-side limit.
- **4.4 Data-at-rest security:** card and limit data are cached locally in encrypted form.
- **4.15 OWASP and build/deploy pipeline:** role/permission checks are enforced consistently at every endpoint against privilege escalation.

Server (localhost simulation): Is the single source of truth for current limits and roles; it never trusts limit data reported by the client.

Extension: Automatic approval thresholds by category (small amounts pass without approval).

21 – ATM Client Simulator



Summary: A desktop application simulating an ATM's card-reading, PIN-entry, and cash-withdrawal steps. An attacker with physical access may try to capture the PIN, replay a transaction, or hack the software to dispense extra cash (a jackpotting simulation).

Key features: Simulated card insertion · PIN entry (secure keypad) · Balance inquiry · Cash withdrawal (limit-checked) · Transaction receipt · Communication with the host

Assets to protect:

- PIN and PIN-block data – **C, I+**
- Transaction session key – **C**
- Withdrawal transaction counter – **I+**

Highlighted requirements:

- **4.14 Binary application protection:** the ATM software is protected against unauthorised DLL injection and process hooking.
- **4.3 Data-in-transit security:** the PIN block and transaction request are sent to the host encrypted and MAC'd.
- **4.10 RASP:** the transaction is aborted and the session terminated when a debugger attachment or memory dump attempt is detected.
- **4.12 Cryptography and certificates:** key management follows a PIN-block encryption scheme.

Server (localhost simulation): The host component holds the balance and daily withdrawal limit; it never approves a transaction without verifying the signature/MAC.

Extension: A cardless (QR/code-based) withdrawal scenario.

22 — QR Payment Client



Summary: A client where a user generates or scans a QR code to make a payment. An attacker may try to capture a QR code and replay it later, or resurrect one whose validity has expired.

Key features: Generating a QR for a payment request · Paying by scanning a QR · Transaction confirmation and receipt · Transaction history · Balance display · Displaying QR validity period

Assets to protect:

- QR payment payload (amount, recipient, timestamp) — **I+**
- Signing/MAC key — **C, I+**
- Transaction history — **I**

Highlighted requirements:

- **4.3 Data-in-transit security:** the QR content and payment confirmation travel to the server over TLS.
- **4.12 Cryptography and certificates:** the QR payload is timestamped and signed; a transaction is never accepted without a valid signature.
- **4.6 Dynamic asset protection:** each QR code can be used only once and only within its validity window; a used or expired code is rejected.
- **4.8 Interface definitions and protection:** the QR payload format (field order, version) is clearly defined and never parsed without validation.

Server (localhost simulation): Marks every QR code as single-use and rejects it on any subsequent submission.

Extension: A static (fixed) QR code driving a variable-amount donation/payment scenario.

23 — Shopkeeper Credit Ledger



Summary: A ledger application in which a shopkeeper tracks the credit (debt) accounts of their customers. The shopkeeper or an assistant may try to alter a past debt record without it being noticed.

Key features: Opening a customer account · Adding a debt/payment entry · Balance display (per customer) · Searching past records · Monthly summary · Backup

Assets to protect:

- Customer debt/payment records — **I+**
- Shopkeeper authentication data — **C, I**
- Backup file — **C, I**

Highlighted requirements:

- **4.4 Data-at-rest security:** records are kept encrypted and integrity-protected in SQLite.
- **4.6 Dynamic asset protection:** a past record can neither be deleted nor overwritten; a correction requires a new offsetting entry (an append-only log).
- **4.7 Asset management:** a policy for regular encrypted backups and retention period is defined.
- **4.1 Development environment security:** no real customer data is used in the development environment; testing relies on synthetic names/amounts.

Server (localhost simulation): Simulates only the periodic upload of the encrypted backup.

Extension: A debt-reminder notification to the customer (simulated local notification).

24 — 📄 Tax Return Preparation Assistant



Summary: An assistant application where a user enters income items and it computes a draft tax return. The financial data entered is highly sensitive; an intercepting party may want both to read the data and to alter the computed result.

Key features: Entering income items (synthetic) · Selecting exemptions/deductions · Automatic tax calculation · Draft return preview · Comparison with the previous year · Export (PDF simulation)

Assets to protect:

- Income and financial data entries — **C**
- Computed result/draft return — **C, I**
- Key derived from the master password — **C**

Highlighted requirements:

- **4.2 Data-in-use security:** the computation happens in memory only; intermediate results are never written to disk in cleartext.
- **4.4 Data-at-rest security:** the draft return is stored encrypted locally.
- **4.15 OWASP and build/deploy pipeline:** input validation (extreme/negative values, injection) is backed by static analysis in the build pipeline.
- **4.13 Certification and penetration test plan:** testing the calculation logic against faulty or malicious inputs is planned.

Server (localhost simulation): Simulates only the retrieval of current (synthetic) tax brackets/rates; it never processes user data.

Extension: Support for additional income types such as freelance or rental income.

25 — Property Deed Document Vault



Summary: A vault application where a user stores a property deed and related documents digitally and shares them, when needed, in a verifiable way. An attacker may want to alter a document's content and assert a fraudulent ownership claim.

Key features: Scanning/adding a document · Categorising documents (deed, permit, insurance) · Verifying a notary/authority signature (simulated) · Sharing a document (verification link) · Viewing version/history · Encrypted archiving

Assets to protect:

- Document files — **C, I+**
- Notary/authority signature certificate chain — **I+**
- Vault encryption key — **C**

Highlighted requirements:

- **4.5 Static asset protection:** document files are kept encrypted on disk and are not directly accessible outside the application.
- **4.12 Cryptography and certificates:** each document's signature chain is verified; the (simulated) root certificate is embedded in the application and kept immutable.
- **4.7 Asset management:** long-term archiving of document versions and access authorisation are planned.
- **4.14 Binary application protection:** the library holding the signature-verification logic is protected against patching that would bypass verification.

Server (localhost simulation): Compares a document's hash/signature record against verification requests arriving via the share link.

Extension: Multi-party approval for a shared-ownership (fractional deed) scenario.

26 — Personal Health Record Vault



Summary: An application where a user keeps synthetic diagnosis, medication and allergy records in a local vault. The attacker is whoever gains hold of the device or finds a lost one; they want to read the records or quietly alter them.

Key features: Add/edit record · View by category (diagnosis/medication/allergy) · Emergency summary card · Encrypted export · Access log · Server synchronisation

Assets to protect:

- Health records (synthetic) — **C, I**
- Emergency summary card — **I**
- Access log — **I+**
- Vault encryption key — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** records held in SQLite with AES-GCM; the key is wrapped and kept in SoftHSM.
- **4.2 Data-in-use security:** the plaintext record is kept in memory only while displayed, for a bounded time.
- **4.7 Asset management:** periodic rotation of the vault key and re-encryption of data under the retired key.
- **4.11 Memory protection:** a viewed record is securely wiped from memory once closed.

Server (localhost simulation): Stores synchronised records and resolves conflicting versions by version number.

Extension: Role-based, partial shared access for family members.

27 – E-Prescription Verifier



Summary: An application a pharmacist uses to scan a physician-signed synthetic prescription and verify its signature and usage status. The attacker wants to forge a prescription or redeem a valid one more than once.

Key features: Prescription scanning (QR/code) · Signature verification · Medication and dosage display · Marking as used (single use) · Verification history · Server-side usage check

Assets to protect:

- Prescription signature – **I+**
- Physician certificate / public key – **I**
- Usage flag – **I+** (must not be reused)
- Patient information (synthetic) – **C**

Highlighted requirements:

- **4.12 Cryptography and certificates:** signature verification with OpenSSL, checking the physician's certificate chain.
- **4.3 Data-in-transit security:** the usage check communicates with the server over TLS 1.3.
- **4.9 Code hardening:** hardened control flow against attempts to bypass the signature-verification step.
- **4.13 Certification and penetration test plan:** a test plan covering forged prescriptions and reuse attempts.

Server (localhost simulation): Checks whether the prescription number has already been used.

Extension: Synchronising usage status across several pharmacies.

28 – Medication Reminder and Dose Log



Summary: An application that records a personal medication schedule and the doses taken. The attacker is whoever can alter the dose log — the user themselves or anyone who gains access to the device — to show a false treatment history.

Key features: Create medication schedule · Reminders · Mark dose taken · View history · Export report · Server backup

Assets to protect:

- Dose log — **I+**
- Medication schedule — **I**
- Exported report — **C, I**
- Backup key — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** dose records are encrypted in SQLite; the key is kept in SoftHSM.
- **4.6 Dynamic asset protection:** the in-memory dose log list is integrity-checked at runtime.
- **4.8 Interface definitions and protection:** the security library's (DLL/.so) logging interface validates its inputs.
- **4.10 RASP:** logging is locked when a debugger attach is detected.

Server (localhost simulation): Stores backups and refuses any rollback of the counter/version number.

Extension: A missed-dose notification for a carer or family member.

29 – Patient Consent Form Signer



Summary: An application for digitally signing a synthetic patient consent form before a procedure. The attacker wants to present an unsigned or later-altered form as if it were still validly signed.

Key features: View form · Capture digital signature · Timestamping · Produce signed document · Verification · Archiving

Assets to protect:

- Signing private key – **C**
- Signed form digest – **I+**
- Timestamp – **I**
- Patient identity (synthetic) – **C**

Highlighted requirements:

- **4.12 Cryptography and certificates:** signing and timestamping with OpenSSL.
- **4.5 Static asset protection:** the form template's integrity is protected, so signing over a tampered template is blocked.
- **4.14 Binary application protection:** the signing function is protected against reverse engineering.
- **4.9 Code hardening:** hardening against debugging/patching attempts aimed at bypassing the signing step.

Server (localhost simulation): Archives signed forms and simulates a timestamp authority role.

Extension: A multi-party signing flow covering both physician and patient.

30 – Lab Result Delivery Client



Summary: An application that delivers synthetic laboratory results end-to-end to a patient or physician. The attacker wants to alter the results or view them without authorisation.

Key features: Receive result · Decrypt and view · Compare against reference range · Notification · Export to PDF · Pull from server

Assets to protect:

- Lab result (synthetic) – **C, I+**
- Patient-result matching key – **C**
- Delivery record – **I**

Highlighted requirements:

- **4.3 Data-in-transit security:** end-to-end encryption; the server carries only ciphertext and cannot read the content.
- **4.4 Data-at-rest security:** the result received on the device is stored encrypted in SQLite.
- **4.7 Asset management:** secure delivery of the decryption key to the patient's device.
- **4.2 Data-in-use security:** the result is decrypted only while being viewed and cleared from memory once the view closes.

Server (localhost simulation): Stores and relays only ciphertext; it cannot decrypt the content.

Extension: Sharing the result with more than one physician, with revocation.

31 – Clinic Appointment Client

Summary: An application for booking and cancelling appointments between patient and clinic. The attacker wants to change someone else's appointment or view the appointment list without authorisation.

Key features: Search appointments · Book/cancel · Reminders · PIN authentication · Appointment history · Server synchronisation

Assets to protect:

- Appointment record – I
- Authentication data (PIN hash) – C, I
- Session token – C

Highlighted requirements:

- **4.2 Data-in-use security:** the plaintext PIN's time in memory during verification is bounded.
- **4.3 Data-in-transit security:** server communication is protected with TLS 1.3 and certificate pinning.
- **4.8 Interface definitions and protection:** the appointment security library's public interface is kept minimal and validated.
- **4.15 OWASP and build/deploy pipeline:** input validation plus static analysis in the build pipeline.

Server (localhost simulation): Resolves appointment clashes on localhost.

Extension: A waiting list and support for several clinic branches.

32 – Vaccination Card Verifier

Summary: An application that verifies synthetic vaccination records via a signed QR code. The attacker wants to forge a vaccination record or alter an existing one.

Key features: QR scanning · Signature verification · View vaccination history · Card issuing for an authorised user · Verification history · Revocation check

Assets to protect:

- Vaccination record signature – I+
- Authority signing key – C, I+
- Revocation list – I

Highlighted requirements:

- **4.12 Cryptography and certificates:** signing/verification with OpenSSL and the authority's certificate chain.
- **4.9 Code hardening:** hardening against attempts to forge or alter the QR content.
- **4.13 Certification and penetration test plan:** a test plan covering forged-card scenarios.
- **4.5 Static asset protection:** the authority's root certificate is kept immutable within the application.

Server (localhost simulation): Publishes revoked records (a revocation-list simulation).

Extension: A multi-dose vaccination schedule and a border-crossing scenario simulation.

 33 – Offline Exam Client

Summary: An application for sitting an exam without an internet connection. The attacker is the student taking the exam themselves; they want to alter answers afterwards, gain extra time, or see the questions in advance.

Key features: Encrypted exam download · View questions · Save answers · Time tracking · Submit results · Tamper detection

Assets to protect:

- Question bank – **C**
- Student answers – **I+**
- Exam timer/counter – **I+** (must not roll back)
- Result file – **I+**

Highlighted requirements:

- **4.10 RASP:** the exam is terminated when a debugger attach or a system-clock rollback is detected.
- **4.4 Data-at-rest security:** questions and answers are encrypted in SQLite with AES-GCM.
- **4.11 Memory protection:** displayed questions and answers are kept in memory only as long as needed.
- **4.6 Dynamic asset protection:** the timer is protected against being rolled back at runtime.

Server (localhost simulation): Collects results once connectivity returns and checks counter consistency.

Extension: Multiple-choice and open-ended questions presented in a mixed, randomised order.

 34 — Exam Question Bank Vault

Summary: A tool where an instructor keeps an encrypted question bank and generates randomised exam papers from it. The attacker is a student sitting the exam, or an insider, wanting to see the questions beforehand.

Key features: Add/edit question · Tag category and difficulty · Generate randomised exam paper · Encrypted export · Password-protected access · Version history

Assets to protect:

- Question bank — **C, I**
- Author credentials — **C**
- Generated exam paper — **C** (until released)

Highlighted requirements:

- **4.4 Data-at-rest security:** the question bank is encrypted in SQLite, with the key wrapped in SoftHSM.
- **4.5 Static asset protection:** sample/template questions shipped with the application are kept unmodifiable.
- **4.1 Development environment security:** real question content must never leak into the repository or build output during development.
- **4.7 Asset management:** lifecycle and access-rights management of the question-bank key.

Server (localhost simulation): Provides encrypted backup only; questions are never stored unencrypted on the server.

Extension: A shared question pool that several instructors contribute to under permission-based access.

 35 — Instructor Gradebook

Summary: An application holding synthetic student grades, whose integrity is critical. The attacker is a student wanting to change their own grade, or anyone who can reach the grade file.

Key features: Grade entry · Weighted average calculation · Per-student report · Grade-change log · Encrypted export · Server backup

Assets to protect:

- Grade records — **I+**
- Change log — **I+**
- Instructor credentials — **C**

Highlighted requirements:

- **4.11 Memory protection:** in-memory grade values are protected against external patching/injection during calculation.
- **4.6 Dynamic asset protection:** the grade list is integrity-checked (checksummed) at runtime.
- **4.9 Code hardening:** hardening against attempts to alter grades with memory-patching tools.
- **4.4 Data-at-rest security:** the gradebook is encrypted in SQLite, with the change log signed separately.

Server (localhost simulation): Stores signed backups and detects rollback attempts.

Extension: Logging a reasoned grade-appeal process.

36 — Diploma and Certificate Verifier



Summary: An application that verifies synthetic diplomas/certificates via a digital signature. The attacker wants to forge a diploma or alter the content of a genuine one.

Key features: Upload/scan document · Signature verification · Institution certificate chain check · Display verification result · History log · Revocation check

Assets to protect:

- Institution signing key — **C, I+**
- Diploma data — **I**
- Revocation list — **I**

Highlighted requirements:

- **4.12 Cryptography and certificates:** certificate chain verification with OpenSSL.
- **4.8 Interface definitions and protection:** the verification library's (DLL/.so) input/output interface is tightly defined.
- **4.13 Certification and penetration test plan:** a test plan covering forged documents and chain manipulation.
- **4.15 OWASP and build/deploy pipeline:** dependency scanning and a signed build output.

Server (localhost simulation): Publishes the revocation list and the root certificate.

Extension: A federated trust network spanning several institutions.

37 – Licensed E-Book Lending System



Summary: An application for borrowing a time-limited e-book from a digital library. The attacker is the reader themselves; they want to exceed the loan period or copy the book's content.

Key features: Search books · Time-limited borrowing · Offline reading · Lock on expiry · Return · Server licence check

Assets to protect:

- Book content key — **C**
- Loan period/counter — **I+** (must not roll back)
- User licence token — **C, I**

Highlighted requirements:

- **4.4 Data-at-rest security:** book content is encrypted with AES-GCM; the key is wrapped in SoftHSM.
- **4.14 Binary application protection:** the reader application is protected against reverse engineering that would leak the content key.
- **4.10 RASP:** the content key is wiped from memory when a debugger is detected.
- **4.6 Dynamic asset protection:** the loan-period counter is protected against being bypassed by rolling back the system clock.

Server (localhost simulation): Limits the number of concurrent loans and refuses attempts to extend the period.

Extension: A waiting list and synchronisation across several devices.

38 — Signed QR Attendance System



Summary: An application that takes classroom attendance using signed, short-lived QR codes. The attacker is a student who missed class; they want to use a classmate's QR code or device to be marked present.

Key features: Timestamped QR generation · QR scanning · Device verification · Attendance list · Export report · Server synchronisation

Assets to protect:

- QR signing key — **C, I+**
- Attendance record — **I+**
- Validity window — **I**

Highlighted requirements:

- **4.12 Cryptography and certificates:** a short-lived, signed QR code generated for each class session.
- **4.3 Data-in-transit security:** replay of the scanned QR is prevented on submission to the server; each QR is single-use.
- **4.9 Code hardening:** hardening against reusing a QR shared as a screenshot.
- **4.7 Asset management:** session keys are invalidated at the end of class.

Server (localhost simulation): Checks the QR code's validity window and uniqueness.

Extension: Concurrent support for several branches/sessions.

39 — Election and Voting Client Simulator



Summary: A small-scale (e.g. a student council) secret-ballot simulation. The attacker wants to alter a vote, or to learn which voter cast which vote.

Key features: Voter authentication · Cast vote · Vote encryption · Tallying · Result display · Audit log

Assets to protect:

- Vote content — **C**
- Voter-vote linkage — **C** (must stay hidden)
- Tally integrity — **I+**
- Audit log — **I+**

Highlighted requirements:

- **4.2 Data-in-use security:** the vote is processed detached from voter identity; the linkage is never held in memory.
- **4.4 Data-at-rest security:** encrypted votes are stored in SQLite and stay unopened individually until tallying.
- **4.12 Cryptography and certificates:** a simplified scheme that only decrypts votes in bulk, at tally time.
- **4.13 Certification and penetration test plan:** a test plan covering repeat voting and voter-linkage attacks.

Server (localhost simulation): Collects encrypted votes and enforces one vote per voter.

Extension: A mechanism letting a voter verify their own vote was counted.

40 — E-Signature Document Signing Tool



Summary: A general-purpose tool for signing documents such as synthetic contracts or petitions. The attacker wants to alter a signed document afterwards while keeping the signature appear valid.

Key features: Upload document · Sign · Multi-signer support · Verification · Timestamp · Archiving

Assets to protect:

- Signing private key — **C**
- Signed document digest — **I+**
- Certificate chain — **I**

Highlighted requirements:

- **4.12 Cryptography and certificates:** signing, verification and timestamping with OpenSSL.
- **4.5 Static asset protection:** root/intermediate certificates are shipped immutably with the application.
- **4.1 Development environment security:** the signing private key must never enter the development/build process.
- **4.14 Binary application protection:** the signing function is protected against extraction or duplication.

Server (localhost simulation): Provides a timestamping and certificate-verification service.

Extension: A sequential multi-signature flow (preparer signs, then approver signs).

41 – Electronic ID Card Reader Simulator



Summary: An application that reads and verifies synthetic e-ID card data (a file-based smart-card simulation). The attacker wants to inject forged card data so it reads as a genuine card.

Key features: Read card data (simulation) · PIN verification · View identity data · Signature verification · Access log · Lock card on wrong PIN

Assets to protect:

- Card private key (simulated) – **C, I+**
- Identity data (synthetic) – **C**
- PIN attempt counter – **I+**

Highlighted requirements:

- **4.12 Cryptography and certificates:** card data is verified against the authority's signature.
- **4.8 Interface definitions and protection:** the card-reader interface is tightly defined and input-validated.
- **4.11 Memory protection:** the PIN is securely wiped from memory after verification.
- **4.10 RASP:** the card session is terminated when tampering or debugging is detected.

Server (localhost simulation): Supplies the root certificate and revocation status for card verification.

Extension: Adding a contactless (NFC simulation) reading scenario.

42 – Citizen Application Form Client



Summary: An application for filling in and submitting an application for a synthetic public service. The attacker wants to apply on someone else's behalf or alter existing application data.

Key features: Fill in form · Attach documents · Submit application · Check status · Notification · Server synchronisation

Assets to protect:

- Application data (synthetic) – **C, I**
- Attached documents – **C, I**
- Application status – **I**

Highlighted requirements:

- **4.3 Data-in-transit security:** the form and attachments are sent over TLS 1.3 with an integrity check.
- **4.7 Asset management:** attachment integrity is tracked by digest (hash) throughout the application.
- **4.15 OWASP and build/deploy pipeline:** input validation, file type/size limits and a secure build pipeline.
- **4.2 Data-in-use security:** identity data is kept in memory for the minimum time while the form is filled in.

Server (localhost simulation): Receives applications and updates status information.

Extension: A multi-step internal approval/routing process.

43 — Certified Document Archive

Summary: An application for archiving synthetic official documents immutably over the long term. The attacker wants to retroactively alter a document in the archive, or its date.

Key features: Upload document · Timestamping · Integrity verification · Search · Version/immutability tracking · Export

Assets to protect:

- Document digest chain (hash chain) — **I+**
- Timestamps — **I+**
- Document content — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** documents and the digest chain are stored encrypted in SQLite.
- **4.12 Cryptography and certificates:** each document digest is chained to the previous one and signed.
- **4.7 Asset management:** version and history records are kept for every document added to the archive.
- **4.13 Certification and penetration test plan:** a test plan covering retroactive alteration (date/digest manipulation) scenarios.

Server (localhost simulation): Simulates a timestamp authority role.

Extension: Publishing the digest chain to a publicly verifiable ledger.

44 — Official Correspondence Client

Summary: An application for preparing and signing official inter-agency correspondence (synthetic, similar to an electronic document management system). The attacker wants to read correspondence content or alter it before it is sent.

Key features: Compose letter · Sign · Send/receive · Reference-number tracking · Archiving · Server synchronisation

Assets to protect:

- Correspondence content — **C, I**
- Signing private key — **C**
- Reference-number sequence — **I+**

Highlighted requirements:

- **4.3 Data-in-transit security:** correspondence is delivered end-to-end encrypted over TLS 1.3.
- **4.12 Cryptography and certificates:** the letter is signed and verified with OpenSSL.
- **4.9 Code hardening:** hardening that makes reading correspondence content out of memory harder.
- **4.8 Interface definitions and protection:** the signing library's public interface is kept minimal and validated.

Server (localhost simulation): Allocates reference numbers and ensures the sequence is never skipped.

Extension: A distinction between normal and classified correspondence.

45 — Passport and Visa Application Tracker



Summary: An application that tracks a synthetic passport/visa application step by step. The attacker wants to alter the application status or the uploaded documents.

Key features: Create application · Upload documents · Appointment tracking · Check status · Notification · Server synchronisation

Assets to protect:

- Application data (synthetic) — **C, I**
- Identity document scan (synthetic) — **C**
- Status history — **I**

Highlighted requirements:

- **4.4 Data-at-rest security:** application data and document scans are stored encrypted in SQLite.
- **4.3 Data-in-transit security:** status checks and document submission are protected with TLS 1.3.
- **4.7 Asset management:** uploaded-document integrity is tracked by digest (hash).
- **4.2 Data-in-use security:** plaintext document data is kept in memory only briefly while viewed.

Server (localhost simulation): Publishes status updates.

Extension: Support for several country/consulate branches.

46 — Employee Badge and Door Access System



Summary: A simulation application for defining and verifying corporate door-access badges. The attacker wants to enter a door they are not authorised for, or to clone a badge.

Key features: Define badge · Assign door/zone rights · Access verification · Access log · Badge revocation · Server synchronisation

Assets to protect:

- Badge key — **C, I+**
- Rights list — **I**
- Access log — **I+**

Highlighted requirements:

- **4.12 Cryptography and certificates:** mutual authentication between badge and reader.
- **4.3 Data-in-transit security:** replay is prevented in reader-badge communication; every request carries a nonce.
- **4.6 Dynamic asset protection:** the rights list is integrity-checked at runtime.
- **4.10 RASP:** access is denied when tampering with the badge simulation file is detected.

Server (localhost simulation): Distributes revoked badges and rights changes.

Extension: Time-window-based access restriction (badge invalid outside working hours).

47 – Visitor Registration and Badge System



Summary: An application that registers an institution's visitors (synthetic data) and admits them with a temporary, signed badge. The attacker wants to forge a badge to enter the building.

Key features: Visitor registration · Add synthetic identity/photo · Signed QR badge issuing · Entry/exit log · Time limit · Server synchronisation

Assets to protect:

- Badge signing key – C, I+
- Visitor data (synthetic) – C
- Entry/exit log – I

Highlighted requirements:

- **4.12 Cryptography and certificates:** every badge is issued time-limited and signed.
- **4.9 Code hardening:** hardening against copying and duplicating the badge QR code.
- **4.4 Data-at-rest security:** visitor records are encrypted in SQLite and auto-purged on expiry.
- **4.13 Certification and penetration test plan:** a test plan covering forged badges and time-extension attempts.

Server (localhost simulation): Tracks valid badges and their expiry.

Extension: Automatic badge pre-approval from a prior appointment.

48 – Single Sign-On (SSO) Client



Summary: A client giving access to several internal applications through a single login. The attacker wants to steal a session token and impersonate the user in another application.

Key features: Authenticate user · Obtain token · Cross-application session sharing · Session renewal · Sign out of all sessions · Token revocation

Assets to protect:

- Session token – C, I+
- User credentials – C
- Refresh key – C

Highlighted requirements:

- **4.3 Data-in-transit security:** the token is delivered over TLS 1.3 with certificate pinning.
- **4.2 Data-in-use security:** the token is kept in memory only briefly and cleared after use.
- **4.8 Interface definitions and protection:** the interface shared across applications is kept minimal and validated.
- **4.11 Memory protection:** the refresh key is not kept in memory unencrypted for an extended period.

Server (localhost simulation): Manages token verification and the revocation list.

Extension: TOTP-based multi-factor authentication integration.

49 — Biometric Template Vault (Simulation)



Summary: A simulation application that stores and matches synthetic, randomly generated biometric template vectors (NOT real fingerprint or face data). The attacker wants to steal a template and replay it on another system.

Key features: Template enrolment (simulated sensor data) · Matching (simulated score) · Template update · Access log · Template revocation/re-enrolment · Server synchronisation

Assets to protect:

- Biometric template (synthetic) — **C, I+**
- Match-score threshold — **I**
- Access log — **I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** the template is encrypted in SQLite, with the key wrapped in SoftHSM.
- **4.2 Data-in-use security:** the template is held in plaintext in memory only during matching, then wiped immediately.
- **4.11 Memory protection:** the matching function is protected against memory dumping.
- **4.14 Binary application protection:** the matching algorithm is protected against reverse engineering that would extract the template.

Server (localhost simulation): Tracks the template's version/revocation status and invalidates the old one on re-enrolment.

Extension: Multi-factor authentication combining a PIN with the simulated biometric.

50 — Corporate Password Policy Checker



Summary: A tool that checks employee passwords against corporate policy and stores their hashes securely. The attacker wants to learn weak passwords or alter the audit reports.

Key features: Define policy · Password strength check · Password history (reuse prevention) · Policy audit report · Check against a local breach list · Server synchronisation

Assets to protect:

- Password hashes (hash + salt) — **C, I**
- Policy rules — **I**
- Audit report — **I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** passwords are stored only as salted hashes in SQLite; the plaintext password is never kept.
- **4.9 Code hardening:** hash comparison is done in constant time to resist timing analysis.
- **4.1 Development environment security:** sample passwords used for testing must not leak into the repository or logs.
- **4.15 OWASP and build/deploy pipeline:** dependency scanning and delivery through a secure build pipeline.

Server (localhost simulation): Distributes company-wide policy rules and collects audit reports.

Extension: Cross-checking against a local, synthetic breach database.

51 – 🎮 Licensed Content Player



Summary: A player that lets a user play purchased audio/video content offline. Content stays encrypted on disk and is decrypted only with a valid licence. The device owner is also a potential attacker: they want to copy the decrypted content from disk or memory, or reuse the licence on another device.

Key features: Content library · Licence activation · Offline playback · Device-bound licence · Playback session · Licence renewal

Assets to protect:

- Content encryption key – **C, I+**
- Decrypted content buffer (in memory) – **C**
- Licence document (device ID, validity) – **C, I+**
- Activation counter – **I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** the content file is encrypted with AES-GCM; the key is wrapped in SoftHSM and bound to the device.
- **4.6 Dynamic asset protection:** content is decrypted piece by piece only at playback time; the full file is never written unencrypted to disk.
- **4.11 Memory protection:** decrypted frames are securely erased after use; screen capture/copy is discouraged where feasible.
- **4.14 Binary application protection:** the player binary is packed/obfuscated and checked against debugger attachment and memory dumping.

Server (localhost simulation): Issues a device-bound licence on purchase and caps the number of activations.

Extension: Playback on a limited number of devices under one licence; remote deactivation when a licence is revoked.

52 — Software Licence Key Manager



Summary: A client library that activates and validates licence keys for a software product. The attacker is the software's own user: they want to forge a valid-looking key, bypass the check, or reuse a single key on many machines.

Key features: Key activation · Machine fingerprinting · Offline validation · Expiry/version check · Licence transfer and revocation

Assets to protect:

- Licence key — **C, I+**
- Signer's public key — **I**
- Machine fingerprint — **I**
- Activation counter — **I+**

Highlighted requirements:

- **4.12 Cryptography and certificates:** the licence is a block signed with the server's private key; the client only ever verifies the signature.
- **4.9 Code hardening:** licence checks are scattered through the code rather than gated by a single `if`, so patching one check is not enough.
- **4.6 Dynamic asset protection:** validity is re-checked periodically at runtime, not just once at start-up.
- **4.13 Certification and penetration test plan:** a test plan covering key sharing and binary-patch bypass scenarios.

Server (localhost simulation): Issues and signs keys, tracks activation counts, and publishes a revocation list.

Extension: A floating licence pool with a concurrent-seat limit for borrowing over the network.

53 — Game Save-File Protector



Summary: A library that protects a game's save files for integrity and confidentiality. The attacker is the player themselves: they want to edit the save to inflate currency, resources, or level, or restore an older save to roll back unwanted progress (rollback abuse).

Key features: Save serialisation · Integrity signature · Encrypted save · Version/rollback detection · Corrupted-save recovery

Assets to protect:

- Save content (progress, inventory) — **I+**
- Save encryption key — **C**
- Integrity MAC — **I**
- Save version counter — **I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** the save file is encrypted with AES-GCM and signed with an HMAC in SQLite.
- **4.9 Code hardening:** key-derivation and integrity-check logic are obfuscated against casual reverse engineering.
- **4.11 Memory protection:** the save encryption key is securely erased from memory after use.
- **4.14 Binary application protection:** the binary is hardened against hex-editing and static patching attempts.

Server (localhost simulation): Keeps the latest valid save version number and rejects an older version being restored.

Extension: Cloud-backed multi-device sync with conflict resolution.

54 – 🛒 In-Game Store Client



Summary: A store client where a player spends in-game currency on content. The attacker is the player: they want to inflate their local balance or tamper with the purchase request to obtain content without paying.

Key features: Product catalogue · Virtual balance display · Purchase flow · Inventory sync · Promotion/discount codes

Assets to protect:

- Virtual balance – **I+**
- Purchase request/response – **I**
- Session token – **C**
- Inventory record – **I+**

Highlighted requirements:

- **4.3 Data-in-transit security:** purchase requests go over TLS 1.3 and are confirmed by a signed server response.
- **4.2 Data-in-use security:** the balance shown locally updates only after server confirmation arrives, never optimistically.
- **4.7 Asset management:** every inventory item is cross-checked and validated against server-side records.
- **4.12 Cryptography and certificates:** purchase requests carry a MAC and a counter, closing the door on replay.

Server (localhost simulation): The single authority for balance and inventory; the client's local values are never trusted alone.

Extension: Time-limited promotion codes with reuse prevention.

55 — 🛡️ Cheat-Resistant Score Submitter



Summary: A client library that reports a game's score to the server. The attacker is the player: they want to submit a forged high score using a memory editor or a modified client.

Key features: Play-session recording · Score computation · Score signing · Server submission · Leaderboard display

Assets to protect:

- Session integrity key — **C**
- Score and gameplay summary — **I+**
- Score-signing key — **C, I+**
- Session ID — **I**

Highlighted requirements:

- **4.10 RASP:** submission is refused when a debugger or memory editor attachment is detected at runtime.
- **4.11 Memory protection:** score and counter variables are obfuscated and checksummed against direct memory writes.
- **4.3 Data-in-transit security:** the score is submitted over TLS, signed with the session key.
- **4.14 Binary application protection:** the client binary is hardened against memory scanners and code injection tools.

Server (localhost simulation): Checks incoming scores against plausibility bounds and flags anomalous submissions.

Extension: Sampling and server-side replay of the recorded gameplay to independently re-derive the score.

56 — Encrypted E-Book Reader



Summary: A reader that lets a user read purchased e-books offline while making it hard to extract the plain text. The attacker is the user: they want to export the decrypted text and share it.

Key features: Library · Licence/key download · Page decryption and rendering · Note-taking · Bookmarks

Assets to protect:

- Book encryption key — **C, I+**
- Decrypted page buffer — **C**
- Licence document — **C, I+**
- User notes — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** the book file is kept AES-GCM-encrypted on disk.
- **4.6 Dynamic asset protection:** only the currently displayed page is decrypted; the whole book is never held open at once.
- **4.5 Static asset protection:** the cover image and embedded fonts are also packaged and integrity-checked.
- **4.11 Memory protection:** the decrypted page buffer is securely erased when the page changes.

Server (localhost simulation): Issues a device-bound key on purchase and binds the downloaded title to the licence.

Extension: A 14-day lending period after which the local copy locks itself automatically.

 57 – Secure Notebook

Summary: A notebook that encrypts and stores a user's private notes locally. The attacker is someone with physical access to the device, trying to read notes past the lock screen.

Key features: Master-password lock · Note creation/editing · Categories/tags · Search over encrypted content · Automatic timed lock

Assets to protect:

- Note content — **C, I**
- Master password hash — **C**
- Note encryption key — **C**
- Search index — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** notes are AES-GCM-encrypted in SQLite; the search index leaves no plain-text trace either.
- **4.2 Data-in-use security:** an open note is held only in memory and its on-screen view is cleared the instant the screen locks.
- **4.11 Memory protection:** the master password and derived key are securely erased after use.
- **4.1 Development environment security:** development and testing never use real personal notes, only synthetic examples.

Server (localhost simulation): Optional; only a simple store that timestamps encrypted backups.

Extension: End-to-end encrypted sync across multiple devices.

58 — End-to-End Encrypted Chat Client



Summary: A chat client that relays messages between two users through a localhost server that cannot read the content itself. The attacker is whoever observes the server or the network in between.

Key features: Key exchange · Message encryption/decryption · Local message history · Identity fingerprint verification · Resend on reconnect

Assets to protect:

- Long-term identity key — **C, I+**
- Session/message key — **C**
- Message content — **C, I**
- Contact fingerprint list — **I**

Highlighted requirements:

- **4.12 Cryptography and certificates:** key exchange and message signatures rely on asymmetric cryptography.
- **4.3 Data-in-transit security:** the server relays ciphertext only and can never decrypt message content.
- **4.2 Data-in-use security:** plaintext messages are held in memory only while displayed.
- **4.11 Memory protection:** session keys are securely erased once a message is processed.

Server (localhost simulation): Relays and queues encrypted messages only; it cannot see the content.

Extension: Forward secrecy (a fresh key per message) protecting past history if a key is later compromised.

59 – Encrypted File Sharing Client



Summary: A client that shares a user's files through a localhost server in encrypted form. The attacker is whoever compromises the server or intercepts the sharing link.

Key features: File encryption · Share link/key generation · Time-limited access · Recipient list · Download and decryption

Assets to protect:

- File encryption key – **C, I+**
- Share access token – **C, I**
- File integrity digest – **I**
- Recipient list – **C**

Highlighted requirements:

- **4.3 Data-in-transit security:** uploads and downloads run over TLS 1.3.
- **4.4 Data-at-rest security:** the file is stored on the server only in encrypted form; the server cannot decrypt it.
- **4.12 Cryptography and certificates:** the share key is asymmetrically wrapped for each recipient.
- **4.7 Asset management:** shared files are tracked in an inventory, and expired shares are deleted automatically.

Server (localhost simulation): Stores the encrypted file and enforces the access token's expiry and download count.

Extension: An additional passphrase layer on the link, defending against whoever intercepts the link itself.



60 – Email Attachment Encryptor



Summary: A tool that encrypts an email attachment before sending so only the intended recipient's key can open it. The attacker is whoever intercepts the email in transit or a message that reaches the wrong recipient.

Key features: Recipient key selection · Attachment encryption · Decryption at the recipient · Passphrase sharing · Attachment integrity check

Assets to protect:

- Attachment content – C, I
- Encryption key/passphrase – C
- Recipient's public key – I
- Integrity signature – I

Highlighted requirements:

- **4.12 Cryptography and certificates:** hybrid encryption – content encrypted symmetrically, key wrapped with the recipient's public key.
- **4.4 Data-at-rest security:** the encrypted attachment stays encrypted on disk until it is sent.
- **4.2 Data-in-use security:** the plaintext attachment exists only in a temporary file, securely deleted after sending.
- **4.8 Interface definitions and protection:** the integration interface with the email client is narrowly and clearly defined.

Server (localhost simulation): None; localhost only simulates the outgoing email queue.

Extension: Separate key wrapping per recipient to support sending to several people at once.

61 – Signed Software Update Client



Summary: A client that downloads and verifies an application's updates from a localhost update server. The attacker is whoever compromises the update channel to distribute a malicious version (a supply-chain attack).

Key features: Version checking · Update download · Signature verification · Rollback prevention · Installation and application

Assets to protect:

- Update-signing public key – I
- Downloaded package – I
- Version number/history – I+
- Installation log – I

Highlighted requirements:

- **4.12 Cryptography and certificates:** every package is signed; the client verifies the signature and certificate chain.
- **4.15 OWASP and build/deploy pipeline:** the signing key lives outside the build pipeline, in a protected environment.
- **4.8 Interface definitions and protection:** the update API follows a strict schema and rejects unexpected fields.
- **4.13 Certification and penetration test plan:** a test plan covering a spoofed update server (man-in-the-middle) scenario.

Server (localhost simulation): A simple version repository serving a signed manifest and package for each release.

Extension: Staged rollout (to a percentage of users first) with automatic rollback on a bad release.

62 – Signed Plugin Loader



Summary: A core that lets the host application load only signed, trusted plugins. The attacker is whoever tries to pass off a malicious plugin as a legitimate one.

Key features: Plugin catalogue · Signature verification · Plugin load/unload · Permission/scope definition · Plugin updates

Assets to protect:

- Plugin-signing key – I
- Plugin binary – I
- Permission/scope definition – I
- Trusted publisher list – I

Highlighted requirements:

- **4.12 Cryptography and certificates:** the plugin's signature and publisher certificate are verified before loading.
- **4.8 Interface definitions and protection:** the plugin API is versioned and exposes a narrow, well-defined set of capabilities.
- **4.9 Code hardening:** the signature check runs early and is obfuscated so it cannot be trivially patched out.
- **4.5 Static asset protection:** plugin packages are stored with an integrity digest in the catalogue.

Server (localhost simulation): An optional plugin catalogue server hosting signed plugins.

Extension: A plugin sandbox restricting which resources a plugin can access.

63 – Configuration Secrets Vault



Summary: A vault library that stores an application's configuration secrets, such as a database password or an API key. The attacker is whoever gets access to the configuration file, the logs, or memory.

Key features: Secret create/update · Environment-scoped configuration (dev/prod) · Secret-read API · Access log · Secret rotation

Assets to protect:

- Secret values — **C, I+**
- Master encryption key — **C**
- Access log — **I**
- Environment tags — **I**

Highlighted requirements:

- **4.4 Data-at-rest security:** secrets are kept encrypted in SQLite and never written to a plain-text file.
- **4.1 Development environment security:** development uses placeholder secrets only; real secrets are never embedded in source.
- **4.7 Asset management:** every secret's lifecycle — creation, rotation, revocation — is tracked.
- **4.11 Memory protection:** the master key is unwrapped in memory only when needed and erased afterwards.

Server (localhost simulation): None; localhost only simulates a rotation scheduler.

Extension: Role-based access control limiting which component may read which secret.



64 — API Developer API Key Manager



Summary: A tool for developers to create and manage their own API keys. The attacker is either someone trying to abuse a stolen key without limit, or a careless developer who leaks a key into logs or a repository.

Key features: Key generation · Scope/permission assignment · Key listing (summary view only) · Key revocation · Usage log

Assets to protect:

- API key — C, I+
- Key hash — I
- Scope definition — I
- Usage log — I

Highlighted requirements:

- **4.7 Asset management:** each key's creation, rotation, and revocation lifecycle is tracked.
- **4.4 Data-at-rest security:** only the key's hash is stored; the raw key is shown once, at creation only.
- **4.1 Development environment security:** keys are never written to logs, version control, or source code.
- **4.8 Interface definitions and protection:** API scopes are defined narrowly, following the principle of least privilege.

Server (localhost simulation): Validates each request against the key hash and rejects out-of-scope requests.

Extension: Per-key rate limiting and alerts on anomalous usage.

65 — Code Signing Tool



Summary: A tool developers use to sign the binaries they build, with the signing key held in a local hardware-security-module simulation. The attacker is whoever wants to steal the signing key to sign a malicious binary.

Key features: Key-pair generation · Binary digest computation · Signing via SoftHSM · Signature verification · Public key/certificate export

Assets to protect:

- Signing private key — C, I+
- Binary digest — I
- Generated signature — I
- Key usage log — I

Highlighted requirements:

- **4.12 Cryptography and certificates:** the private key lives inside [SoftHSM/PKCS#11](#) and never leaves it.
- **4.1 Development environment security:** signing runs only on an authorised, network-isolated build machine.
- **4.13 Certification and penetration test plan:** key-theft and unauthorised-signing scenarios are part of the test plan.
- **4.7 Asset management:** every signed binary's inventory entry and timestamp are recorded.

Server (localhost simulation): None; localhost only offers a timestamping simulation.

Extension: Multi-signature (two developers' approval) for signing critical releases.

66 — Encrypted Versioned Backup Client



Summary: A client that sends a user's files to a localhost backup server in encrypted form and keeps a version history. The attacker is whoever compromises the backup server or the transfer.

Key features: File selection · Incremental backup · Encryption · Version history and restore · Integrity verification

Assets to protect:

- Backup encryption key — C, I+
- Backup content — C, I
- Version metadata — I
- Integrity digest — I

Highlighted requirements:

- **4.4 Data-at-rest security:** backup chunks on the server are end-to-end encrypted.
- **4.3 Data-in-transit security:** backup traffic is protected with TLS 1.3.
- **4.7 Asset management:** the version list and retention policy are tracked.
- **4.11 Memory protection:** the backup key stays in memory only for the duration of the transfer.

Server (localhost simulation): Stores encrypted backup chunks and the version list; it cannot decrypt the content.

Extension: A 3-2-1 backup strategy simulation with a second local copy.

 67 – Secure Clipboard Manager

Summary: A tool that keeps an encrypted history of sensitive clipboard data (such as passwords) and clears it automatically after a set time. The attacker is another application or user on the same device.

Key features: Clipboard history log · Timed auto-clear · Sensitive-data flagging · Search over encrypted history · Manual deletion

Assets to protect:

- Clipboard history content – **C**
- History encryption key – **C**
- Sensitive-data tags – **C**
- Access log – **I**

Highlighted requirements:

- **4.2 Data-in-use security:** sensitive clipboard data is held in memory for the shortest time possible before being cleared.
- **4.11 Memory protection:** every in-memory copy of a clipboard value is securely erased.
- **4.4 Data-at-rest security:** the clipboard history is stored encrypted in SQLite.
- **4.9 Code hardening:** the sensitive-data detection logic is hardened against simple bypass attempts.

Server (localhost simulation): None; localhost only simulates optional multi-device sync.

Extension: Per-application clipboard access permission, so only authorised applications can read the history.

68 — Document Watermarker



Summary: A tool that embeds a visible and an invisible watermark (user identity, timestamp) into shared documents so a leak can be traced back to its source. The attacker is whoever strips the watermark and shares the document without authorisation.

Key features: Document upload · Visible watermark · Robust invisible watermark embedding · Watermark extraction and verification · Document inventory

Assets to protect:

- Watermark key/seed — C, I
- Document content — C
- Watermark-to-user mapping — I
- Document inventory — I

Highlighted requirements:

- **4.5 Static asset protection:** distributed documents are protected as watermarked static assets with an integrity digest.
- **4.7 Asset management:** who received which document, and when, is tracked.
- **4.9 Code hardening:** the embedding/extraction logic is made difficult to reverse.
- **4.6 Dynamic asset protection:** a user-specific watermarked copy is generated on the fly for every download.

Server (localhost simulation): Generates a user-specific watermarked copy on every document request and logs the mapping.

Extension: Using the extraction tool to trace a leaked document back to its source.

69 — Encrypted Photo Gallery



Summary: A gallery application that stores a user's photos locally in encrypted albums. The attacker is whoever gets access to the device or intercepts a backup file.

Key features: Photo import · Encrypted storage · Albums/tagging · Thumbnail preview · Secure deletion

Assets to protect:

- Photo files — **C**
- Thumbnail cache — **C**
- Gallery encryption key — **C**
- Album metadata — **C**

Highlighted requirements:

- **4.4 Data-at-rest security:** photos and thumbnails are kept AES-GCM-encrypted.
- **4.5 Static asset protection:** the thumbnail cache is also encrypted, leaving no plain-text trace behind.
- **4.11 Memory protection:** the decoded image buffer is securely erased after viewing.
- **4.6 Dynamic asset protection:** a preview is decrypted only while being viewed, never decrypted in bulk.

Server (localhost simulation): None; localhost only supports an optional encrypted backup simulation.

Extension: A time-limited, separately keyed sharing link for album sharing.

70 — Encrypted Voice Memo Recorder



Summary: An application that records and encrypts a user's voice memos. The attacker is whoever gets access to the device or intercepts a shared memo over the network.

Key features: Audio recording · Encrypted storage · Playback · Note/tag attachment · Sharing over localhost

Assets to protect:

- Voice recording file — **C**
- Recording encryption key — **C**
- Note/tag metadata — **C**
- Share access token — **C, I**

Highlighted requirements:

- **4.4 Data-at-rest security:** recordings are kept AES-GCM-encrypted on disk.
- **4.2 Data-in-use security:** the raw audio buffer while recording lives only in memory and is never written unencrypted.
- **4.11 Memory protection:** the audio buffer is securely erased once recording finishes.
- **4.3 Data-in-transit security:** sharing happens over TLS with a time-limited access token.

Server (localhost simulation): Holds a shared recording encrypted and temporarily, deleting it once the token expires.

Extension: An offline, local speech-to-text simulation producing an automatic note summary.



71 – Time-Limited Licensed Content Downloader



Summary: A client that lets a user download and watch licensed content for a limited window (e.g. 48 hours). The attacker is the user who wants to keep using the content after the window expires.

Key features: Content download · Time-limited licence · Countdown/expiry check · Automatic deletion on expiry · Re-download under a fresh licence

Assets to protect:

- Content encryption key – **C**, **I+**
- Licence expiry time – **I+**
- Downloaded content – **C**
- Device clock reference – **I**

Highlighted requirements:

- **4.6 Dynamic asset protection:** the expiry check is re-verified against the server's timestamp at every playback.
- **4.3 Data-in-transit security:** download and licence renewal are protected with TLS.
- **4.12 Cryptography and certificates:** the licence is signed, so its expiry field cannot be altered by the client.
- **4.14 Binary application protection:** the binary is hardened against patching out the expiry check.

Server (localhost simulation): Issues content and a time-limited licence, producing a server-timestamped, signed expiry.

Extension: A reminder before expiry and one-click re-rental.

72 — Offline Map Package Licence Checker



Summary: A client that enforces the licence on offline map packages a user has downloaded. The attacker is the user who wants to unlock the whole map without buying a given region's package.

Key features: Region package catalogue · Package download · Licence-to-region matching · Offline map display · Package updates

Assets to protect:

- Map package file — I
- Region licence record — I+
- Package integrity digest — I
- Licence-package signature — I

Highlighted requirements:

- **4.5 Static asset protection:** map packages are protected with an integrity digest; a tampered package is rejected.
- **4.6 Dynamic asset protection:** only a licensed region unlocks; other regions stay locked.
- **4.12 Cryptography and certificates:** the licence-to-package match is distributed signed.
- **4.13 Certification and penetration test plan:** package-tampering and region-boundary-bypass scenarios are part of the test plan.

Server (localhost simulation): Issues a signed package and licence for a purchased region.

Extension: Separate licensing of sub-layers within a package, such as roads versus points of interest.

73 — Game Cheat Detection Library



Summary: A library embedded in a game that detects memory editors, debuggers, and code-injection tools. The attacker is the player: they want to alter values with a memory scanner or inject code.

Key features: Process/memory integrity scanning · Debugger detection · Known cheat-tool signature matching · Violation reporting · Safe game termination

Assets to protect:

- Detection signature database — **I**
- Integrity baseline (reference digest) — **I**
- Violation log — **I**
- Detection parameters — **C**

Highlighted requirements:

- **4.10 RASP:** tampering is detected at runtime and answered immediately, such as ending the session.
- **4.11 Memory protection:** the library's own detection variables are protected against tampering too.
- **4.9 Code hardening:** the detection logic is obfuscated against reverse engineering.
- **4.8 Interface definitions and protection:** the interface between the game and the library is kept narrow enough to resist abuse.

Server (localhost simulation): Collects violation reports and flags an account after repeated violations.

Extension: Behavioural anomaly detection (unusual speed/hit rate) supplementing signature-based detection.

74 — Digital Event Ticket Wallet



Summary: A wallet that stores a user's purchased event tickets as QR codes, verified at entry. The attacker is the user who wants to duplicate a ticket (by screenshot) to let several people in at once.

Key features: Ticket purchase · QR code generation · Offline viewing · Entry-point verification · Ticket transfer

Assets to protect:

- Ticket-signing key — **C, I+**
- Ticket ID and usage status — **I+**
- QR code content — **I**
- Transfer record — **I**

Highlighted requirements:

- **4.12 Cryptography and certificates:** the ticket is signed, so a forged QR code cannot be produced.
- **4.6 Dynamic asset protection:** the QR code refreshes at short intervals, making a shared screenshot far less useful.
- **4.3 Data-in-transit security:** entry-point verification queries the server over TLS.
- **4.14 Binary application protection:** the ticket data is protected against tampering on the client side.

Server (localhost simulation): Tracks ticket usage status and accepts a given ticket only once, rejecting double use.

Extension: Instant invalidation of the old QR code the moment a ticket is transferred.

75 — Time-Limited Film Rental Client



Summary: A client that lets a user rent a film for a fixed window (e.g. a 48-hour viewing window starting from first playback). The attacker is the user who wants to bypass the time limit and watch indefinitely.

Key features: Film catalogue · Rental purchase · Window start on first playback · Countdown · Access revocation on expiry

Assets to protect:

- Film encryption key — **C, I+**
- Rental start/end time — **I+**
- Decrypted frame buffer — **C**
- Licence document — **C, I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** the film file is kept encrypted on disk.
- **4.6 Dynamic asset protection:** the rental window's start matches the server's; a changed local clock cannot extend it.
- **4.11 Memory protection:** decrypted frames are short-lived in memory, erased right after use.
- **4.14 Binary application protection:** the binary is hardened against patching out the window/playback check.

Server (localhost simulation): Records the first-playback event and returns the rental window's end time, signed.

Extension: Offline tracking of the rental window with a mandatory periodic online re-check.

76 — 📱 Smart Lock Control Client



Summary: A desktop client that locks and unlocks a smart door lock over Bluetooth/Wi-Fi; no real lock hardware is involved, the lock is represented server-side as a software simulation. An attacker who seizes the device or eavesdrops on the traffic wants to unlock the door without authorisation or hide the access record.

Key features: Device pairing · Unlock/lock with PIN or passcode · Temporary guest code generation · Access history view · Remote lock-status query

Assets to protect:

- Unlock command and pairing key — **C, I+**
- PIN/guest code — **C, I**
- Access (lock/unlock) history — **I+** (must not be deleted or rewritten)
- Device identity certificate — **I**

Highlighted requirements:

- **4.3 Data-in-transit security:** the unlock command is sent encrypted over TLS with a timestamp; replay is prevented.
- **4.12 Cryptography and certificates:** every device has its own pairing certificate; a spoofed device is rejected.
- **4.10 RASP:** an attached debugger or memory tampering is detected and the unlock command is refused.
- **4.4 Data-at-rest security:** PINs and keys are stored encrypted in SQLite, never in plain text.

Server (localhost simulation): Holds lock state and the list of paired devices; rejects commands from a spoofed or unpaired device.

Extension: Time-limited access grants for multiple users (e.g. a cleaner allowed only between 09:00–11:00).

77 – 🏠 Smart Home Gateway Configurator



Summary: A configuration client that manages in-home IoT devices (lamp, socket, sensor simulations) through a single gateway; no real hardware is involved, devices are simulated as local processes/localhost endpoints. An attacker who breaches the network wants to alter the device list or add a rogue device.

Key features: Device discovery and pairing · Wi-Fi credential provisioning · Scene/automation rule definition · Firmware update trigger · Device status monitoring

Assets to protect:

- Wi-Fi credentials — **C**
- Device pairing keys — **C, I+**
- Firmware update package — **I+** (unsigned packages are rejected)
- Device inventory/configuration — **I**

Highlighted requirements:

- **4.7 Asset management:** each device's identity, key, and last-seen time are tracked in an inventory; lost devices are revoked.
- **4.8 Interface definitions and protection:** the gateway-device protocol is versioned; undefined commands are rejected.
- **4.3 Data-in-transit security:** Wi-Fi credential provisioning uses an end-to-end encrypted channel.
- **4.1 Development environment security:** test keys never leak into the production build.

Server (localhost simulation): Holds paired device identities and the firmware signing key; rejects unsigned updates.

Extension: A separate trust zone that limits device visibility for the guest network.

78 — 🧠 Smart Electricity Meter Data Collector



Summary: A client that collects periodic consumption readings from a smart electricity meter (software simulation) and forwards them to the billing server. The homeowner is also a plausible attacker who wants to under-report consumption to avoid part of the bill.

Key features: Periodic reading · Local cache buffering · Batched upload to server · Meter authentication · Post-outage data integrity check

Assets to protect:

- Consumption readings — **I+** (must not be rewritten retroactively; the meter must not roll back)
- Meter identity certificate — **I**
- Transmission session key — **C**
- Local cache file — **C, I**

Highlighted requirements:

- **4.4 Data-at-rest security:** the local cache is encrypted with AES-GCM using a key bound to the meter identity.
- **4.12 Cryptography and certificates:** every reading upload is signed with the meter's certificate.
- **4.9 Code hardening:** the reading calculation logic is obfuscated so it cannot easily be patched.
- **4.3 Data-in-transit security:** upload to the server uses TLS 1.3 with mutual authentication.

Server (localhost simulation): Holds the last reading for each meter identity; rejects a rolled-back or unsigned reading.

Extension: A monitoring dashboard that alerts on missed transmission windows.

79 – EV Charging Station Client



Summary: A client that starts and stops an EV charging station (software simulation) and computes the energy consumed and the fee; no real charging hardware is involved. A malicious user wants to under-report the charging duration/amount to avoid paying.

Key features: User authentication · Charging session start/stop · Live power/energy indicator · Billing with a payment token · Session receipt

Assets to protect:

- Charging session data (kWh, duration) – I+
- Payment token – C
- Station identity certificate – I
- Session receipt – I+

Highlighted requirements:

- **4.3 Data-in-transit security:** station-server communication is protected by TLS with mutual certificate verification.
- **4.2 Data-in-use security:** the payment token is kept unencrypted in memory only during the transaction, then wiped.
- **4.12 Cryptography and certificates:** the session receipt is signed with the station's key.
- **4.13 Certification and penetration test plan:** a dedicated penetration-test scenario is defined for the payment flow.

Server (localhost simulation): Holds open sessions and consumed energy; rejects an unsigned or inconsistent close request.

Extension: A station-fleet view that manages several charging points at once.

80 — 🚗 Vehicle Diagnostics (OBD) Data Logger



Summary: A client that reads fault codes and mileage from a vehicle's OBD interface (software simulation, no real vehicle is connected) and logs them in signed form. An owner wanting to roll back the mileage before selling the vehicle is a plausible attacker.

Key features: Fault code (DTC) reading · Mileage logging · Service history view · Report export · Device authentication

Assets to protect:

- Mileage record — I+ (cannot be decreased retroactively)
- Fault code history — I+
- OBD device identity — I
- Exported report — I+

Highlighted requirements:

- **4.14 Binary application protection:** the mileage calculation logic is hardened against reverse engineering.
- **4.9 Code hardening:** checksums guard against in-memory value tampering (e.g. a cheat engine).
- **4.4 Data-at-rest security:** records are stored signed and encrypted in SQLite, protected by a strictly increasing counter.
- **4.11 Memory protection:** the mileage value is kept in a secure memory region during processing.

Server (localhost simulation): Holds the last reported mileage per vehicle identity; rejects a decreasing value.

Extension: Chaining the service history with hash chains to verify integrity.

81 — 🚚 Fleet Tracking with Signed GPS Logs



Summary: A client that periodically signs and forwards fleet vehicles' (software simulation) location data to a central server. A driver may want to alter the location log to hide off-duty use or a route deviation.

Key features: Periodic location sampling · Local signed log keeping · Batched synchronisation to the server · Route history view · Device-driver pairing

Assets to protect:

- GPS location log — **I+** (signed, sequence must not be broken)
- Device signing key — **C, I+**
- Driver identity — **C**
- Synchronisation session key — **C**

Highlighted requirements:

- **4.12 Cryptography and certificates:** every location entry is signed with the device's private key; the key is kept wrapped in SoftHSM.
- **4.4 Data-at-rest security:** the local log is encrypted and protected by an increasing sequence number; entries cannot be spliced.
- **4.3 Data-in-transit security:** synchronisation uses TLS 1.3 and resumes where it left off after an outage.
- **4.7 Asset management:** the vehicle-to-key mapping is kept in a central inventory; a lost device is revoked.

Server (localhost simulation): Holds the last sequence number per vehicle; rejects an unsigned or sequence-skipping entry.

Extension: A simple geofence check that raises an automatic alert on off-route excursions.

82 — Digital Tachograph Simulator



Summary: A software simulation of a digital tachograph recording driver working/rest times; no real vehicle hardware is involved. A driver may want to rewrite past records to avoid exceeding the permitted driving time.

Key features: Driver card (simulated) sign-in · Driving/rest state logging · Daily summary report · Record export · Inspection mode view

Assets to protect:

- Driving/rest time records — **I+** (cannot be rewritten retroactively)
- Driver card data (simulated) — **C**
- Inspection report — **I+**
- Device clock/timestamp — **I**

Highlighted requirements:

- **4.14 Binary application protection:** integrity checking against tampering with the app via a debugger/patch.
- **4.10 RASP:** tampering is detected, logging is suspended, and the event is flagged.
- **4.12 Cryptography and certificates:** the daily summary is signed with the device certificate.
- **4.4 Data-at-rest security:** records are stored append-only, each entry's signature chained to the previous one.

Server (localhost simulation): Stores the daily summaries submitted per driver; detects and rejects a broken chain.

Extension: A simple inspection dashboard that alerts on excessive driving-time violations.

83 — Parcel Delivery Verification System



Summary: A client-server system in which a courier confirms delivery with signed proof (recipient signature/code simulation). Either the courier or the recipient may want to misrepresent whether or how a delivery occurred to create a dispute.

Key features: Delivery list view · Recipient identity/code verification · Delivery proof capture (signature/photo simulation) · Status update · Delivery history query

Assets to protect:

- Delivery proof — **I+** (signed, cannot be altered afterwards)
- Recipient verification code — **C**
- Courier identity certificate — **I**
- Delivery status log — **I+**

Highlighted requirements:

- **4.12 Cryptography and certificates:** delivery proof is signed with the courier device's private key.
- **4.3 Data-in-transit security:** status updates are sent over TLS with a timestamp.
- **4.8 Interface definitions and protection:** the courier-server API is versioned; invalid status transitions are rejected.
- **4.7 Asset management:** courier device identities and keys are kept in a central list and can be revoked.

Server (localhost simulation): Holds the current status per shipment identity; rejects an unsigned or inconsistent status transition.

Extension: Letting the recipient confirm delivery with a second signature from their own device (dual proof).

84 — Warehouse Stock Management with Access Control



Summary: A desktop application through which warehouse staff, by role (clerk/auditor/manager), record stock in and out. A low-privilege clerk may want to alter a stock record in their own favour or escalate their privileges.

Key features: Role-based sign-in · Stock in/out recording · Stocktake (inventory) report · Approval chain for critical operations · Audit log view

Assets to protect:

- Stock quantity records — I+
- User role/permission data — C, I
- Audit log — I+
- Approval-chain signatures — I+

Highlighted requirements:

- **4.8 Interface definitions and protection:** every operation is constrained by role-based authorisation, re-checked server-side.
- **4.4 Data-at-rest security:** stock and audit data is encrypted in SQLite and cannot be altered by direct file editing.
- **4.9 Code hardening:** client-side permission checks are also enforced server-side (the client is never trusted alone).
- **4.7 Asset management:** the user-role mapping is kept centrally; an account is revoked immediately on departure.

Server (localhost simulation): Holds the role/permission table; rejects an unauthorised operation and an unsigned approval.

Extension: A simple rule engine that suggests reordering when a critical quantity is reached.

85 — 📡 Cold-Chain Temperature Logger



Summary: A logging device client that periodically measures temperature during transport (sensor simulation) and keeps a signed log; no real sensor is involved. A carrier may want to hide a temperature excursion to avoid liability.

Key features: Periodic temperature sampling · Threshold-breach alarm · Signed log keeping · Shipment report export · Device calibration info view

Assets to protect:

- Temperature log — **I+** (cannot be rewritten retroactively)
- Calibration certificate — **I**
- Alarm threshold configuration — **I**
- Device signing key — **C, I+**

Highlighted requirements:

- **4.12 Cryptography and certificates:** every log entry is signed with the device key and chained.
- **4.10 RASP:** a case-opening signal (simulated) flags tampering without halting the log.
- **4.4 Data-at-rest security:** the log file is stored encrypted and append-only in SQLite.
- **4.6 Dynamic asset protection:** the alarm threshold configuration cannot be altered without authorisation at runtime.

Server (localhost simulation): Holds the log chain per shipment identity; rejects a broken or unsigned segment.

Extension: Merging data from multiple sensors into a single shipment report.

86 — Industrial PLC Configuration Tool (Simulation)



Summary: An engineering-station client that prepares and uploads ladder-logic configuration for an industrial PLC (software simulation, no real device involved). An unauthorised person altering the configuration could trigger a dangerous state on the production line.

Key features: Configuration project creation/editing · Engineer authentication · Uploading the configuration to the device (simulation) · Version history · Change approval

Assets to protect:

- PLC configuration file — I+
- Engineer credentials — C
- Deployment log — I+
- Signed configuration package — I+

Highlighted requirements:

- **4.1 Development environment security:** configuration projects are version-controlled and built signed; the engineer's machine is kept isolated.
- **4.5 Static asset protection:** the upload package is protected by an integrity digest; an altered package is not deployed (simulation) to the device.
- **4.8 Interface definitions and protection:** the engineering station-PLC protocol is restricted to defined commands.
- **4.13 Certification and penetration test plan:** a dedicated test plan is defined for the industrial control system scenario.

Server (localhost simulation): Verifies the signature and version of uploaded configurations; rejects an unsigned upload.

Extension: A four-eyes approval flow requiring two engineers' sign-off for critical changes.

87 – SCADA Alarm Client (Simulation)



Summary: A client that monitors industrial-plant alarm events (software simulation) and shows them to an operator, who can acknowledge/clear them; no real SCADA infrastructure is involved. Unauthorised access might hide a critical alarm to delay response.

Key features: Operator sign-in · Live alarm list · Alarm acknowledgement/clearing · Event history query · Priority/threshold configuration

Assets to protect:

- Alarm event stream – I+
- Operator credentials – C
- Alarm acknowledgement/clearing log – I+
- Threshold configuration – I

Highlighted requirements:

- **4.3 Data-in-transit security:** the alarm stream is delivered over TLS through a low-latency, verified channel.
- **4.8 Interface definitions and protection:** the alarm acknowledgement command is accepted only from an authorised operator session.
- **4.2 Data-in-use security:** operator credentials are protected in memory for the duration of the session and never leak into a screenshot.
- **4.13 Certification and penetration test plan:** a dedicated penetration-test scenario is defined given the critical-infrastructure simulation.

Server (localhost simulation): Holds the alarm queue and acknowledgement state; rejects an unauthorised acknowledgement request.

Extension: An automatic escalation rule for critical alarms that remain unacknowledged.



88 — Medical Device Configuration Simulator



Summary: A client that prepares dosage/parameter configuration for a medical device (e.g. an infusion-pump simulation); no real device or real patient data is involved — all patient data is **synthetic**. Because an unauthorised change would endanger patient safety, access control and integrity are critical.

Key features: Clinician authentication · Dosage/parameter configuration · Safe-range limit checking · Sending the configuration to the device (simulation) · Change history

Assets to protect:

- Dosage/parameter configuration — **I+**
- Clinician credentials — **C**
- Synthetic patient record — **C, I**
- Configuration change log — **I+**

Highlighted requirements:

- **4.13 Certification and penetration test plan:** a regulatory-style penetration-test plan is prepared for medical device software.
- **4.9 Code hardening:** the safe dosage-range check is enforced independently on both the client and the server.
- **4.10 RASP:** an attached debugger is detected and configuration submission is blocked.
- **4.4 Data-at-rest security:** the synthetic patient record and configuration history are kept encrypted.

Server (localhost simulation): Holds the last approved configuration per device identity; rejects a value outside the safe range.

Extension: Requiring a second clinician's approval for critical parameter changes.

89 — 🚁 Drone Flight Plan Signer



Summary: A client that prepares and digitally signs a flight plan for an unmanned aerial vehicle (drone, software simulation); no real aircraft is involved. A pilot or an unauthorised party might secretly alter a plan to fly outside the permitted zone.

Key features: Flight route drawing/editing · No-fly-zone checking · Plan signing · Telemetry log view (simulation) · Plan history query

Assets to protect:

- Flight plan — **I+** (signed, cannot be altered without approval)
- Pilot credentials — **C**
- Signing key — **C, I+**
- Telemetry log (simulation) — **I+**

Highlighted requirements:

- **4.12 Cryptography and certificates:** the flight plan is signed with the pilot's private key; the server verifies the signature.
- **4.3 Data-in-transit security:** plan and telemetry transmission is protected by TLS.
- **4.14 Binary application protection:** the no-fly-zone check logic is hardened against bypass.
- **4.8 Interface definitions and protection:** the plan-submission API accepts only signed and verified plans.

Server (localhost simulation): Holds approved flight plans and their signatures; rejects a plan that enters a no-fly zone or lacks a signature.

Extension: Generating an automatic cancellation signal (simulation) when an in-flight route deviation is detected.

90 — 📺 Security Camera Footage Archive



Summary: A system that archives footage from security cameras (software simulation) and lets it be reviewed according to access rights. An unauthorised person might delete, alter, or watch footage without permission; evidentiary integrity (chain of custody) matters.

Key features: Camera recording ingest (simulated stream) · Archiving and tagging · Role-based viewing rights · Export (evidence package) · Access log

Assets to protect:

- Video recordings — C, I+
- Access/viewing log — I+
- Exported evidence package — I+
- Camera identity certificates — I

Highlighted requirements:

- **4.4 Data-at-rest security:** recordings are stored on disk encrypted with AES-GCM, the key wrapped in SoftHSM.
- **4.7 Asset management:** who accessed which recording and when is tracked in an inventory.
- **4.13 Certification and penetration test plan:** a dedicated test plan is defined for the chain-of-custody scenario.
- **4.3 Data-in-transit security:** camera-to-archive transmission is protected by TLS with mutual authentication.

Server (localhost simulation): Holds recording integrity digests and the access log; rejects an inconsistent export request.

Extension: Adding an independently verifiable timestamp to an exported evidence package.

91 – P Car Park Entry-Exit System



Summary: A car park client that manages entry/exit via plate/ticket reading (simulation) and computes the fee. A user or an attendant might under-report the parked duration or duplicate a ticket to avoid paying.

Key features: Entry ticket generation · Exit-time duration/fee calculation · Payment capture (simulation) · Barrier-open command · Entry-exit log

Assets to protect:

- Ticket identity and entry time – I+
- Payment transaction data – C, I
- Barrier-open command – I+
- Entry-exit log – I+

Highlighted requirements:

- **4.9 Code hardening:** the duration/fee calculation logic is hardened so it cannot be trivially altered client-side.
- **4.8 Interface definitions and protection:** the barrier-open command is generated only after a valid payment approval and cannot be invoked directly.
- **4.6 Dynamic asset protection:** ticket identities are generated unpredictably; a duplicated ticket is rejected.
- **4.4 Data-at-rest security:** the entry-exit log is stored encrypted and immutable.

Server (localhost simulation): Holds open tickets and payment state; rejects a reused or unpaid ticket.

Extension: Automatic pass-through for monthly-subscription plates via a separate authorisation flow.

92 — Hotel Key Card Encoder Simulator



Summary: A client used at a hotel front desk to encode a guest's door card (RFID simulation) with time-limited access; no real card hardware is involved. Someone who breaches the front-desk software might want to produce a card valid past checkout, or for a different room.

Key features: Reservation matching · Card encoding (room + duration) · Card cancellation/re-encoding · Master-card operations · Card usage log

Assets to protect:

- Card access key — C, I+
- Master key — C, I+
- Room-duration configuration — I
- Card usage log — I+

Highlighted requirements:

- **4.12 Cryptography and certificates:** the card key is derived from the room and validity period; the master key is kept wrapped in SoftHSM.
- **4.4 Data-at-rest security:** the encoding history and key-derivation parameters are stored encrypted in SQLite.
- **4.14 Binary application protection:** the card-encoding logic is protected so reverse engineering cannot expose the master key.
- **4.10 RASP:** an attached debugger is detected and master-key operations are halted.

Server (localhost simulation): Holds the room/duration for active cards; rejects an expired or cancelled card.

Extension: Broadcasting an instant revocation signal to all locks (simulation) when a card is reported lost.



93 – Ψ Restaurant Order and Kitchen Display with Access Control



Summary: An order system, role-authorised, where waiters enter orders and the kitchen sees them. An unauthorised user might alter an order or a payment amount to commit embezzlement (internal theft).

Key features: Role-based sign-in (waiter/kitchen/manager) · Order creation/cancellation · Kitchen display synchronisation · Bill closing/payment · Discount/cancellation approval chain

Assets to protect:

- Order and bill records – **I+**
- Discount/cancellation approval log – **I+**
- Payment link token – **C**
- User role data – **C, I**

Highlighted requirements:

- **4.8 Interface definitions and protection:** cancellation/discount operations are accepted only from an authorised role and re-verified server-side.
- **4.2 Data-in-use security:** the payment token is kept unencrypted in memory only at the moment of payment.
- **4.7 Asset management:** staff accounts and roles are kept in a central list; sessions close at the end of a shift.
- **4.9 Code hardening:** price/discount calculation is hardened against client-side manipulation.

Server (localhost simulation): Holds open bills and the approval chain; rejects an unauthorised discount/cancellation request.

Extension: Comparing expected versus actual amounts through an end-of-shift Z report.

 94 – Payroll Vault

Summary: A vault application that stores employee salary information and payroll history, accessible only to authorised HR/accounting staff. An insider with access might want to alter their own salary or view others' payroll data without authorisation.

Key features: Authorised staff sign-in · Payroll record entry/view · Salary calculation (deduction/tax simulation) · Payment instruction with an approval chain · Access/view log

Assets to protect:

- Salary/payroll records – **C, I**
- Employee credentials – **C**
- Payment instruction – **I+**
- Access log – **I+**

Highlighted requirements:

- **4.4 Data-at-rest security:** payroll records are field-level encrypted in SQLite, decryptable only with the authorised key.
- **4.2 Data-in-use security:** salary data is shown on screen only within an authorised session and only as much as needed.
- **4.11 Memory protection:** temporary values used during salary calculation are securely wiped after processing.
- **4.7 Asset management:** who may access which payroll is defined in a role-based inventory.

Server (localhost simulation): Holds approved payment instructions; rejects an unauthorised or unsigned instruction.

Extension: A four-eyes flow requiring a second authoriser's approval for salary changes.

95 – Corporate VPN Configuration Client



Summary: A configuration client that lets employees connect to the corporate network over VPN; it connects not to a real VPN server but to a localhost simulation server. An attacker in possession of a stolen laptop might try to use the VPN credentials to breach the network.

Key features: User authentication (password + simulated second factor) · Configuration profile download · Connect/disconnect · Certificate renewal · Connection log

Assets to protect:

- VPN credentials – C
- Client certificate and private key – C, I+
- Configuration profile – I
- Connection log – I+

Highlighted requirements:

- **4.12 Cryptography and certificates:** the client certificate is kept wrapped in SoftHSM; the server verifies it via mutual TLS.
- **4.3 Data-in-transit security:** all connection setup and traffic run over TLS 1.3.
- **4.1 Development environment security:** test/development certificates never mix into the production configuration.
- **4.5 Static asset protection:** the configuration profile file is protected by an integrity signature; an altered profile is rejected.

Server (localhost simulation): Holds valid certificates and active sessions; rejects an expired or revoked certificate.

Extension: A simple risk rule requiring extra verification for a connection attempt from a suspicious location.

96 – Security Event Log Collector Agent



Summary: An agent that collects security event logs from various components (application/system simulation) and forwards them tamper-evidently to a central server. An attacker who has breached the system wants to erase or alter log records to cover their tracks.

Key features: Log-source monitoring · Local buffer accumulation · Signed batch upload · Delivery acknowledgement (ack) tracking · Resuming after an outage

Assets to protect:

- Raw log records – I+ (append-only, cannot be altered)
- Agent identity certificate – I
- Transmission sequence/counter – I+
- Local buffer – C, I

Highlighted requirements:

- **4.4 Data-at-rest security:** the local buffer is stored encrypted and append-only; retroactive editing is prevented.
- **4.9 Code hardening:** the agent binary is hardened against a patch attempt meant to hide or clear logs.
- **4.6 Dynamic asset protection:** code injection into the agent process is detected at runtime.
- **4.15 OWASP and build/deploy pipeline:** the agent's build pipeline enforces dependency scanning and signed releases.

Server (localhost simulation): Holds the last accepted sequence number per agent; rejects a sequence-skipping or unsigned batch.

Extension: A "silence" (missed heartbeat) alert for agents that stop reporting within the expected window.



97 – Counterfeit Part Verifier



Summary: A client that reads a digitally signed serial number on an industrial/automotive spare part (software simulation) and verifies the manufacturer's signature. A counterfeit-part producer might try to imitate or copy a genuine signature.

Key features: Part serial-number reading (barcode/code simulation) · Signature verification · Manufacturer record lookup · Suspicious/counterfeit part flagging · Scan history

Assets to protect:

- Manufacturer verification certificate – I
- Part serial number and signature – I+
- Scan history – I+
- Verification key – C, I+

Highlighted requirements:

- **4.12 Cryptography and certificates:** every part is signed with the manufacturer's private key; the client verifies with the public key only.
- **4.5 Static asset protection:** the verification key/certificate store is protected by an integrity check against tampering.
- **4.14 Binary application protection:** the verification logic is hardened against a fake "always valid" patch.
- **4.8 Interface definitions and protection:** the scan-result API returns only defined verification codes.

Server (localhost simulation): Holds known serial numbers and their signature status; rejects a previously revoked part.

Extension: A duplicate-part alert that detects the same serial number scanned in multiple locations.

98 — Laboratory Sample Chain of Custody



Summary: A chain-of-custody system that signs and logs every hand-off of a laboratory sample, from intake to the results report; all patient-related data is **synthetic** only. An unauthorised person might alter a sample record to hide a step in the chain.

Key features: Sample intake and labelling (simulated barcode) · Hand-off recording · Technician authentication · Result entry and reporting · Chain history query

Assets to protect:

- Chain-of-custody records — **I+** (signed, chronological)
- Synthetic patient/sample data — **C, I**
- Technician credentials — **C**
- Results report — **I+**

Highlighted requirements:

- **4.12 Cryptography and certificates:** each hand-off record is signed with the relevant technician's key and appended to the chain.
- **4.4 Data-at-rest security:** chain records are stored append-only, encrypted, in SQLite.
- **4.13 Certification and penetration test plan:** because the system requires forensic/legal validity, a dedicated verification and penetration-test plan is defined.
- **4.7 Asset management:** technician identities and permission scopes are kept in a central list.

Server (localhost simulation): Verifies chain integrity per sample identity; rejects a broken or unsigned link.

Extension: Exporting every step of the chain with an independently verifiable digest (hash).

99 – Smart Farming Sensor Gateway



Summary: A system that collects data from field humidity/temperature sensors (software simulation) through a gateway and forwards it to a central server; no real sensor is involved. An attacker who breaches the gateway might inject spoofed sensor data to disrupt irrigation/fertilisation decisions.

Key features: Sensor registration/pairing · Periodic reading collection · Local cache buffering · Batched transmission to the server · Sensor status monitoring

Assets to protect:

- Sensor reading data – I
- Sensor pairing keys – C, I+
- Gateway firmware – I+
- Local cache – C, I

Highlighted requirements:

- **4.3 Data-in-transit security:** sensor-to-gateway and gateway-to-server transmission use an encrypted channel.
- **4.7 Asset management:** registered sensors' identity and key inventory is kept; data from an unrecognised sensor is rejected.
- **4.6 Dynamic asset protection:** a spoofed sensor registration attempt is detected at runtime.
- **4.1 Development environment security:** gateway firmware is produced through a signed build pipeline.

Server (localhost simulation): Holds registered sensor identities and their latest readings; rejects data from an unpaired sensor.

Extension: A simple anomaly alert for abnormal reading patterns (sudden spikes).

100 — Personal Key and Certificate Manager



Summary: A manager application that keeps a user's personal private keys, certificates, and passphrases in a single secure vault and brokers signing/decryption operations; it distils, on its own, the key-management thinking that underlies the other 99 projects of the term. An attacker who seizes the device wants to capture all the keys.

Key features: Key-pair generation · Certificate request (CSR) creation and import · Sign/decrypt brokering (the private key never leaves the vault) · Key/certificate renewal and revocation · Encrypted backup

Assets to protect:

- Private keys — C, I+
- Certificates — I
- Master passphrase/key — C, I+
- Backup file — C, I+

Highlighted requirements:

- **4.12 Cryptography and certificates:** all key generation/storage goes through [SoftHSM/PKCS#11](#); the private key never appears in cleartext in application memory.
- **4.4 Data-at-rest security:** certificate metadata and the backup file are kept encrypted with AES-GCM.
- **4.11 Memory protection:** intermediate values used during signing are securely wiped after processing.
- **4.14 Binary application protection:** the application is hardened so reverse engineering cannot expose the master passphrase/key.
- **4.10 RASP:** an attached debugger is detected, the vault locks, and operations are refused.

Server (localhost simulation): Holds the certificate revocation list (CRL simulation) and renewal requests; rejects an expired certificate.

Extension: Encrypted synchronisation of the key vault across multiple devices.